INTENT RECOGNITION FOR CODE-MIXED SINHALA-ENGLISH HOUSEHOLD SERVICE REQUESTS USING MULTILINGUAL SENTENCE EMBEDDINGS AND SUPPORT VECTOR MACHINES

Individual Thesis

BANDARA W A M S

(IT22551320)

Individual component: Intelligent Service Discovery and Intent Recognition
(Component 01)
Group ID: R26-IT-090

Dissertation submitted in partial fulfilment of the requirements for the
Bachelor of Science (Hons) in Information Technology
Specializing in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology
Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this is my own work and this dissertation does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the non-exclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy (Appendix B). I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name	Student ID	Signature	Date
BANDARA W A M S	IT22551320		10/27/2026

Supervisor certification. The above candidate has carried out research for the bachelor's degree dissertation under my supervision.

Role and name	Signature	Date
Supervisor: Prof. Dasuni Nawinna		10/27/2026
Co-supervisor: Mr. Nelum Chathuranga Amarasena		10/27/2026

ABSTRACT

Customers of household-service platforms in Sri Lanka are usually asked to choose a trade category before they can find a worker. Many customers do not know the correct trade name, and they describe their problems in informal, code-mixed language, often Sinhala written in Latin letters and mixed with English ("Singlish"). Existing Sinhala-English research focuses on social-media classification and transliteration, and existing Sinhala intent classifiers cover small chatbot intent sets. No published work maps romanised Singlish household problem descriptions to fine-grained worker categories. This thesis presents the intent recognition component of an AI-powered household service marketplace. The component converts one free-text problem description, in English or romanised Singlish, into one of 39 worker categories. A balanced dataset of 21,294 descriptions (546 per category, about 42% romanised Singlish) was built from team-written phrases, problems derived from public Facebook groups, and AI-assisted generation. Descriptions are encoded without pre-processing using the pretrained multilingual sentence-embedding model paraphrase-multilingual-MiniLM-L12-v2. Two decision methods were compared on the same stratified 80/20 split. FAISS cosine nearest-neighbour search achieved 71.50% accuracy (macro-F1 0.71). An RBF-kernel support vector machine achieved 78.68% accuracy (macro-F1 0.79), improving F1 in 36 of 39 categories. After ambiguous carpenter rows were cleaned, a separate run reached 78.35% accuracy (macro-F1 0.78). Remaining errors cluster in closely related trades, such as carpenter and furniture restoration and bathroom and kitchen water heaters. The model is served through a FastAPI service that rejects predictions below 0.4 confidence. It is deployed on Microsoft Azure App Service and integrated with the Node.js backend, the Flutter app and the recommendation component. Because most of the data is AI-generated, the results describe performance on synthetic descriptions; validation on real customer queries is the main next step.

Keywords: code-mixed Sinhala-English, intent classification, multilingual sentence embeddings, support vector machine, service marketplace

ACKNOWLEDGEMENTS

I am sincerely grateful to my supervisor, Prof. Dasuni Nawinna, and my co-supervisor, Mr. Nelum Chathuranga Amarasena, for their guidance, encouragement and constructive feedback throughout this research. Their advice shaped both the research questions and the practical design of the component.

I thank the members of group R26-IT-090 for their collaboration in building the marketplace application, the backend and the other intelligent components, and for their help in preparing seed phrases and integrating the intent recognition service. I also thank the lecturers and staff of the Department of Information Technology at the Sri Lanka Institute of Information Technology for the knowledge and facilities that made this work possible.

I acknowledge the members of public Sri Lankan online community groups whose openly shared household problems helped us understand how people describe repair needs in everyday language. I acknowledge the use of an AI assistant for generating part of the training data and for drafting assistance, as disclosed in Appendix B.

Finally, I thank my family and friends for their patience and constant support.

TABLE OF CONTENTS

DECLARATION AND SUPERVISOR CERTIFICATION i

ABSTRACT ii

ACKNOWLEDGEMENTS iii

LIST OF ABBREVIATIONS xi

1 INTRODUCTION 1

1.1 Overall Project Context 1

1.2 Individual Component Context 2

1.3 Problem Statement 3

1.4 Research Gap 3

1.5 Aim and Objectives 4

1.6 Research Questions 5

1.7 Scope, Limitations and Assumptions 5

1.8 Individual Contribution 6

1.9 Thesis Structure 7

2 LITERATURE REVIEW AND RELATED WORK 8

2.1 Review Method and Scope 8

2.2 Domain and Theoretical Foundations 8

2.2.1 Code-mixing and romanised Sinhala 8

2.2.2 Intent classification 9

2.2.3 Sentence embeddings 9

2.2.4 Support vector machines and probability estimates 10

2.2.5 Vector similarity search 10

2.3 Related Research 11

2.3.1 Sinhala-English code-mixed text classification 11

2.3.2 Language identification and transliteration 11
2.3.3 Sinhala pretrained language models 12
2.3.4 Sinhala intent classification and chatbots 12
2.3.5 Code-mixed intent detection in other languages 12
2.3.6 Service-request and ticket classification 13
2.3.7 Synthetic training data for intent classification 13
2.4 Existing Systems, Products and Techniques 13
2.5 Gap Synthesis and Conceptual Framework 15
3 RESEARCH METHODOLOGY 17
3.1 Research Design 17
3.2 Data, Participants and Study Materials 19
3.2.1 Category taxonomy 19
3.2.2 Data sources and construction 19
3.2.3 Dataset profile 20
3.2.4 Cleaned dataset for Experiment B 21
3.3 Experimental Procedure 22
3.3.1 Loading and splitting 22
3.3.2 Sentence encoding 22
3.3.3 Approach 1: FAISS similarity search 23
3.3.4 Approach 2: SVM classifier 23
3.3.5 Experiment B: cleaned dataset 24
3.4 Measures and Evaluation Metrics 24
3.5 Data Analysis 24
3.6 Ethics, Privacy, Security and Responsible AI 25
3.7 Reproducibility and Research Artefacts 26
4 PRODUCT AND SOLUTION DESIGN 27

4.1 Requirements 27
4.2 Component Architecture within the Complete Solution 28
4.3 Design Alternatives and Decisions 29
4.4 Detailed Design 29
4.4.1 Inference flow 29
4.4.2 Decision rule 30
4.4.3 Interface contract 31
4.4.4 Module structure 31
4.5 Standards and Professional Practice 32
5 IMPLEMENTATION, INTEGRATION AND TESTING 33
5.1 Development Environment and Technology Stack 33
5.2 Component Implementation 33
5.2.1 Data loading and splitting 33
5.2.2 Encoding 33
5.2.3 FAISS baseline 34
5.2.4 SVM training 34
5.2.5 Inference function 34
5.3 Integration with the Other Components 35
5.4 Verification and Software Testing 35
5.5 Deployment and Operation 37
6 RESULTS AND VALIDATION 38
6.1 Research Results 38
6.1.1 Experiment A: overall comparison 38
6.1.2 Experiment A: per-category results 39
6.1.3 Experiment B: SVM on the cleaned dataset 40
6.1.4 Error analysis 41

6.2 Product and Technical Results 44
6.2.1 Functional behaviour 44
6.2.2 Sample queries 44
6.2.3 Efficiency 45
6.3 Integration Results 45
6.4 Summary of Findings 47
7 DISCUSSION 48
7.1 Interpretation of Findings 48
7.2 Comparison with Literature Baselines and Existing Solutions 49
7.3 Research Contribution 50
7.4 Product Contribution 51
7.5 Limitations and Threats to Validity 51
7.6 Implications 53
8 COMMERCIALISATION AND SUSTAINABILITY 54
8.1 Value Proposition and Target Users 54
8.2 Market and Competitive Position 54
8.3 Business and Sustainability Model 55
8.4 Scalability, Intellectual Property and Compliance 56
8.5 Contribution to the Sustainable Development Goals 56
9 CONCLUSION AND FUTURE DIRECTIONS 58
9.1 Conclusion 58
9.2 Achievement of Objectives 58
9.3 Individual Research and Product Contributions 59
9.4 Future Directions 59
REFERENCES 61
GLOSSARY 66

APPENDICES 67
Appendix A: Category List and Extended Results 67
Appendix B: Academic Integrity and AI Use Disclosure 70

LIST OF FIGURES

Figure 1.1: Overall architecture of the smart service marketplace and the position of Component 01 2

Figure 2.1: Conceptual framework: positioning of this study relative to existing research and platforms 16

Figure 3.1: Research workflow followed for Component 01 18

Figure 3.2: Dataset composition: (a) estimated language of each description, (b) distribution of description length by language 21

Figure 4.1: Internal architecture of the Component 01 service and its connection to the backend and mobile tiers 28

Figure 4.2: Inference flow of Component 01, including the 0.4 confidence rule 30

Figure 4.3: Request and response formats of the Component 01 prediction endpoint 31

Figure 6.1: Experiment A: accuracy and macro-averaged precision, recall and F1-score of the two approaches 39

Figure 6.2: Experiment A: per-category F1-score of FAISS top-1 search and the SVM classifier, sorted by SVM F1 (dashed lines show the macro averages) 40

Figure 6.3: Experiment A: row-normalised confusion matrix of the SVM classifier (rows = actual category, columns = predicted category) 42

Figure 6.4: Experiment A: most frequent confusions of the SVM classifier, as a share of each true category's test samples 43

LIST OF TABLES

Table 2.1: Comparison of existing worker-finding platforms with the proposed component 14

Table 2.2: Candidate technique families for Component 01 15

Table 3.1: Profile of the balanced dataset used in Experiment A 20

Table 3.2: Representative rows from the dataset 21

Table 3.3: Configuration of the final model 23

Table 4.1: Functional requirements of Component 01 27

Table 4.2: Non-functional requirements of Component 01 27

Table 4.3: Design decisions for Component 01 29

Table 5.1: Technology stack of Component 01 33

Table 5.2: Test cases for Component 01 36

Table 6.1: Experiment A - overall results on the same 4,259-row test set 38

Table 6.2: Experiment B – SVM results on the cleaned dataset (3,885-row test set) 41

Table 6.3: Sample queries passed through the trained SVM (notebook output) 44

Table 9.1: Achievement of research objectives 58

Table A.1: The 39 worker categories and the estimated share of romanised Singlish rows per category (balanced dataset, 546 rows each) 67

Table A.2: Experiment A: per-category precision (P), recall (R) and F1-score of FAISS top-1 search and the SVM classifier (test support 109–110 per category) 68

Table B.1: Disclosure of AI tool use 70

LIST OF ABBREVIATIONS

Abbreviation	Description
AI	Artificial Intelligence
API	Application Programming Interface
ASR	Automatic Speech Recognition
BERT	Bidirectional Encoder Representations from Transformers
CI	Confidence Interval
CPU	Central Processing Unit
CSV	Comma-Separated Values
FAISS	Facebook AI Similarity Search
FN	False Negative
FP	False Positive
FR	Functional Requirement
GPU	Graphics Processing Unit
HTTP(S)	Hypertext Transfer Protocol (Secure)
IEC	International Electrotechnical Commission
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
k-NN	k-Nearest Neighbours
LaBSE	Language-agnostic BERT Sentence Embedding
mBERT	Multilingual BERT
ML	Machine Learning
NFR	Non-Functional Requirement
NIST	National Institute of Standards and Technology
NLP	Natural Language Processing
NLU	Natural Language Understanding
PDPA	Personal Data Protection Act (Sri Lanka, No. 9 of 2022)
RBF	Radial Basis Function
RQ	Research Question
SBERT	Sentence-BERT
SDG	Sustainable Development Goal
SLIIT	Sri Lanka Institute of Information Technology
SVM	Support Vector Machine
TF-IDF	Term Frequency-Inverse Document Frequency

Abbreviation	Description
TP	True Positive
XLM-R	XLM-RoBERTa (cross-lingual RoBERTa language model)

1 INTRODUCTION

1.1 Overall Project Context

Household maintenance in Sri Lanka depends on a large, informal workforce of plumbers, electricians, carpenters, masons, appliance technicians, cleaners and many other tradespeople. Most of these workers are not represented by firms, brands or formal listings. Customers usually find them through relatives, neighbours, notice boards, classified advertisements and social media groups. This arrangement works when a trusted contact is available. It breaks down when a household faces an unfamiliar problem, has recently moved, or simply does not know what kind of worker is needed.

The group project, "AI-Powered Smart Service Marketplace for Household Maintenance in Sri Lanka" (Group R26-IT-090), addresses this customer-side problem with a mobile application backed by four intelligent components. Figure 1.1 shows how these components fit together.

- **Component 01 – Intelligent Service Discovery and Intent Recognition** (this thesis) lets a customer describe a problem in their own words, in English or romanised Sinhala-English ("Singlish"), and converts that description into a structured worker category.
- **Component 02 – Hybrid Worker Recommendation** filters available workers by category and ranks them using a machine-learning estimate of historical suitability, adjusted by live distance and live ratings, and returns the top twenty candidates.
- **Component 03 – Hybrid Sentiment and Trust Scoring** analyses verified post-job reviews written in English or Singlish and fuses their sentiment with star ratings to produce a category-specific trust score, which Component 02 consumes as its rating input.
- **Component 04 – Fair Price Prediction** is planned to estimate a fair price for a requested job so that customers and workers can agree on a reasonable charge.

The customer-facing application is built with Flutter [1]. A Node.js backend coordinates requests, and the machine-learning components are exposed as Python services using FastAPI [2].

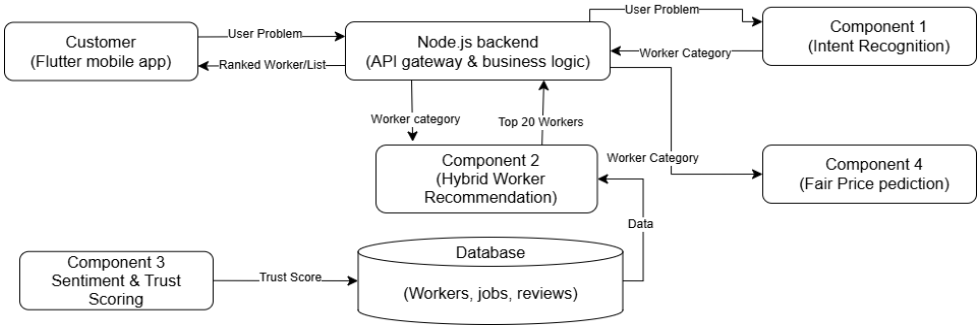


Figure 1.1: Overall architecture of the smart service marketplace and the position of Component 01

1.2 Individual Component Context

Component 01 is the "front door" of the marketplace. Every customer journey begins with a search, and every later component depends on the category that this search produces. If the category is wrong, Component 02 searches the wrong pool of workers, Component 03's trust scores are looked up for the wrong trade, and the customer receives irrelevant recommendations.

Conventional service applications avoid language understanding altogether by asking customers to pick a category from a menu. That design assumes that the customer already knows the correct trade name. In practice, Sri Lankan customers describe symptoms rather than trades, and they often mix Sinhala and English in Latin script. Examples include "watura bata pupurala" (the water pipe has burst), "current eka wada na" (the electricity is not working) and "AC eka sisil wenne na" (the air conditioner is not cooling). A customer who types "geyser eke pressure relief valve drip" is describing a bathroom water heater problem, yet may not know whether to look for a plumber, an electrician or a water-heater technician.

The component therefore takes one free-text problem description and returns one of 39 worker categories, together with a confidence score. The implemented pipeline encodes the raw text with a pretrained multilingual sentence-embedding model, paraphrase-multilingual-MiniLM-L12-v2 [3], [4]. It then classifies the resulting 384-

dimensional vector with a support vector machine (SVM) using a radial basis function (RBF) kernel [5]. If the highest class probability is below 0.4, the application tells the customer that no suitable worker could be identified instead of guessing.

1.3 Problem Statement

Customers of household-service platforms in Sri Lanka must translate an everyday problem into a formal trade category before they can find a worker. This is difficult for three reasons.

1. **Vocabulary mismatch.** Customers describe what they observe ("the sink water is not going down", "roof leaks when it rains") rather than the trade that fixes it (drain unblocking, roof repair, waterproofing). Some trades are closely related, for example bathroom versus kitchen water heaters, or carpentry versus furniture restoration, which makes manual selection error-prone.
2. **Code-mixed, romanised input.** A large share of informal written communication in Sri Lanka uses Sinhala words written in Latin characters, mixed with English words, with no standard spelling. Keyword search over English category names cannot match such text.
3. **Fine-grained categories.** A useful marketplace needs many specific trades. The dataset prepared for this project defines 39 categories, from AC technician to well cleaner. That is much larger than the small intent sets found in existing Sinhala chatbots.

The research problem addressed in this thesis is therefore: *how can free-text household problem descriptions written in English or romanised Sinhala-English be automatically and reliably mapped to one of 39 worker categories, in a form that can be integrated into a mobile marketplace?*

1.4 Research Gap

Research on Sinhala-English code-mixed text has focused mainly on social-media classification tasks: sentiment, hate speech, humour and aspect identification [6], [7]. It has also covered word-level language identification [8] and transliteration of romanised Sinhala into Sinhala script [9]. Existing Sinhala intent classifiers target

Sinhala-script or bilingual chatbots with small intent sets in domains such as restaurant booking [10], general conversational assistants [11] and spoken commands [12]. Service-request classification has been studied for English IT help-desk tickets [13], [14]. Code-mixed task-oriented language understanding has been studied mainly for Hindi-English and other Indian language pairs [15], [16].

On the product side, global platforms such as TaskRabbit ask customers to choose a task category, and their own support pages acknowledge that "not every task will fall neatly into one category" [17]. Sri Lankan platforms such as ikman.lk, work.lk and HodaBass.lk organise listings by category and keyword [18], [19], [20]. They therefore assume that the customer already knows the correct worker type.

To the best of the author's knowledge, no published work classifies free-text household problem descriptions written in romanised Sinhala-English into fine-grained worker categories, and no such dataset exists. Section 2.5 develops this gap in detail.

1.5 Aim and Objectives

The aim of this research is to design, implement and evaluate an intent recognition component that maps free-text household problem descriptions in English and romanised Singlish to one of 39 worker categories, and to integrate it with the marketplace application.

The specific objectives are to:

1. Review existing research on code-mixed Sinhala-English processing, intent classification and service-request classification, and identify the gap that this component addresses.
2. Build a labelled dataset of English and romanised Singlish household problem descriptions covering 39 worker categories, balanced across categories.
3. Design a classification pipeline that does not require hand-crafted Singlish spelling rules, using pretrained multilingual sentence embeddings as features.
4. Compare a similarity-search approach (FAISS nearest neighbour) with a trained classifier (SVM) on the same embeddings, data split and test set.

5. Evaluate the selected model using accuracy, precision, recall, F1-score and per-category error analysis.
6. Expose the model as a service with a confidence-based rejection rule and integrate it with the Node.js backend, the Flutter application and Component 02.

1.6 Research Questions

The study is guided by the following research questions.

- **RQ1.** Can a pretrained multilingual sentence-embedding model, used without fine-tuning, provide features that distinguish 39 household worker categories in English and romanised Singlish text?
- **RQ2.** Does a discriminative classifier (RBF-kernel SVM) trained on these embeddings outperform nearest-neighbour similarity search (FAISS top-1) on the same data?
- **RQ3.** Which worker categories are most often confused, and what does this reveal about the category design and the dataset?
- **RQ4.** Can the resulting model be integrated into a mobile marketplace as a service that returns a single worker category, and declines to answer when its confidence is low?

1.7 Scope, Limitations and Assumptions

In scope. The component accepts a single free-text description typed by the customer. It supports English and romanised Sinhala-English (Sinhala words written in Latin letters, mixed with English). It predicts exactly one of the 39 worker categories listed in Appendix A, Table A.1, and returns a confidence score. It rejects queries whose highest probability is below 0.4.

Out of scope. The following are outside the scope of this component:

- Sinhala script, Tamil script and romanised Tamil-English ( ) input.
- Prediction of a specific issue type within a category (for example "pipe repair" within plumbing).
- Multi-label requests that need two trades at once.

- Explicit spelling correction, phonetic matching or transliteration of Singlish words.
- Fine-tuning of the embedding model.

Spell correction and fine-tuning are discussed as future work in Chapter 9.

Assumptions and limitations. The main assumptions and limitations are:

- Most of the dataset was generated with the help of an AI assistant, seeded with phrases written by the team and with problems re-written from public Facebook group posts. Labels were assigned at generation time and were not independently annotated. The reported accuracy therefore describes performance on this largely synthetic data, not on real customer queries.
- The service is deployed on a single Microsoft Azure App Service instance (Basic B1 plan), which suits a research prototype rather than production-scale traffic.
- User-level validation with real customers was not carried out in this study.

These limitations are discussed in Section 7.5.

1.8 Individual Contribution

The author was responsible for the whole of Component 01, specifically:

- Defining the 39-category taxonomy and the dataset format (problem_description, worker_type).
- Preparing the dataset, including writing seed phrases, collecting and re-writing problems from public Facebook groups, directing AI-assisted generation, cleaning duplicates and ambiguous rows, and balancing classes.
- Designing and implementing the two modelling approaches (FAISS similarity search and SVM classification) in Google Colab.
- Evaluating both approaches and analysing classification errors.
- Implementing the inference service and the 0.4 confidence rule, and deploying the service on Microsoft Azure.
- Defining the output contract (worker_type) that Component 02 consumes.

Integration with the mobile application and backend was carried out together with the other group members.

1.9 Thesis Structure

The remainder of the thesis is organised as follows:

- Chapter 2 reviews related research and existing products and synthesises the research gap.
- Chapter 3 describes the research design, the dataset, the experimental procedure and the evaluation metrics.
- Chapter 4 presents the requirements and the design of the component within the overall solution.
- Chapter 5 describes the implementation, integration and testing.
- Chapter 6 reports the results.
- Chapter 7 discusses their meaning, limitations and contribution.
- Chapter 8 considers commercialisation and sustainability.
- Chapter 9 concludes and outlines future directions.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review Method and Scope

The review was organised around the three questions that shape the design of Component 01:

1. How is code-mixed Sinhala-English text processed?
2. How are user utterances mapped to intents or categories?
3. How do existing service platforms help a customer reach the right worker?

Sources were identified through the ACL Anthology, IEEE Xplore, Springer Link, arXiv, the institutional repositories of the University of Moratuwa and the University of Colombo School of Computing, and the documentation of the models and tools used. Search terms combined "Sinhala", "Singlish", "code-mixed", "code-switched", "romanised" and "transliteration" with "intent", "classification", "chatbot", "sentence embedding" and "ticket".

Peer-reviewed papers from 2017 onwards were preferred, together with foundational papers on support vector machines and sentence embeddings. Preprints and product documentation were used only where no peer-reviewed source covered the point, and they are identified as such in the text.

2.2 Domain and Theoretical Foundations

2.2.1 Code-mixing and romanised Sinhala

Code-mixing (or code-switching) is the use of two or more languages within a single utterance or conversation. It is a normal feature of bilingual communities and is widespread in informal written text [21]. Surveys of the field show that most computational work concentrates on a few language pairs, notably Hindi-English and Spanish-English, and that resources for other pairs remain scarce [22]. Sinhala is classed as a low-resource language: few large annotated corpora and tools are available, and those that exist mostly target Sinhala script [23], [24].

In informal digital communication, Sri Lankans often write Sinhala words with Latin characters and mix them with English words. A request such as "kitchen eke sink eka

block wela" combines Sinhala grammar and function words ("eke", "eka", "wela") with English nouns ("kitchen", "sink", "block"). Romanised Sinhala has no standard spelling: the same word may appear as "watura", "wathura" or "watr". This makes word-level matching unreliable and is the main reason why approaches built for Sinhala script cannot be applied directly [8], [9].

2.2.2 Intent classification

Intent classification assigns an utterance to one of a fixed set of goals and is a core part of task-oriented dialogue and search systems. Benchmarks such as CLINC150 [25], BANKING77 [26] and the multilingual MASSIVE corpus [27] provide 60 to 150 intents and have become standard references. Two findings from this literature shape the present work.

- Classifiers that perform well on in-scope intents can still be poor at recognising queries that belong to none of the intents. Out-of-scope handling must therefore be designed explicitly [25].
- Fixed pretrained sentence encoders combined with a light classifier can match or exceed fully fine-tuned BERT models for intent detection, especially when training data is limited. They also train in minutes on a CPU [26].

2.2.3 Sentence embeddings

Transformer language models such as BERT [28] and its multilingual variant XLM-RoBERTa (XLM-R) [29] produce contextual token representations. Sentence-BERT (SBERT) fine-tunes such models in a siamese configuration, so that mean-pooled token vectors become sentence embeddings whose cosine similarity reflects semantic similarity [30].

Reimers and Gurevych later extended SBERT to many languages through knowledge distillation [3]. A multilingual student model is trained so that the embedding of a translated sentence matches the English teacher's embedding of the original sentence. The model used in this thesis, paraphrase-multilingual-MiniLM-L12-v2, is a student of this kind. It has a 12-layer MiniLM architecture [31], roughly 0.1 billion parameters (most of them in the embedding table of its large multilingual vocabulary), 384-

dimensional output vectors, mean pooling and a maximum input length of 128 tokens [4].

Two properties of this model matter for Singlish:

- It uses the XLM-R SentencePiece tokenizer [32], [29]. Any string, including misspelled or romanised words, is broken into sub-word pieces rather than mapped to an "unknown" token.
- The model card lists about fifty languages for the paraphrase training. Sinhala is not among them [4]. The model has therefore never been trained to align Sinhala, let alone romanised Sinhala, with English meaning.

Section 7.1 returns to the consequences of this point.

2.2.4 Support vector machines and probability estimates

Support vector machines find a maximum-margin decision boundary between classes [5]. With a kernel such as the radial basis function (RBF), they can learn non-linear boundaries in the original feature space. SVMs were shown early on to be strong text classifiers because they cope well with high-dimensional feature spaces [33]. They remain competitive baselines in ticket and help-desk classification [13], [14].

For multi-class problems, the widely used LIBSVM implementation [34], which scikit-learn wraps [35], trains one binary classifier per pair of classes. Class probabilities are obtained by fitting a sigmoid to the decision values (Platt scaling) [36] and combining the pairwise estimates (pairwise coupling) [37]. These probabilities are what make a confidence-based rejection rule possible. Rejecting predictions whose maximum class probability is low is a simple and established baseline for detecting misclassified and out-of-distribution inputs [38].

2.2.5 Vector similarity search

FAISS is a library for efficient similarity search over dense vectors [39]. When vectors are L2-normalised, an inner-product index returns the training examples with the highest cosine similarity to a query. Using the label of the nearest training example (top-1 nearest neighbour) is the simplest way to turn a sentence-embedding model into

a classifier. It needs no training beyond building the index and is a natural baseline against which to compare a trained classifier.

2.3 Related Research

2.3.1 Sinhala-English code-mixed text classification

Rathnayake et al. [6] built a manually annotated corpus of 10,000 Sinhala-English code-mixed and code-switched user comments covering five tasks: sentiment analysis, humour detection, hate speech detection, language identification and aspect identification. They showed that adapter-based fine-tuning of pretrained multilingual language models performs as well as, or better than, full fine-tuning on this data and on Kannada-English and Hindi-English datasets. Rizvi et al. [7] fine-tuned XLM-R for aspect-based sentiment analysis of Sinhala and code-mixed banking reviews, adding a domain lexicon correction step, and reported 88.4% on Sinhala and code-mixed content. That work is currently a preprint.

Both studies classify opinions and social-media content into a small number of affective or topical labels. Neither addresses service requests or worker categories.

2.3.2 Language identification and transliteration

Smith and Thayasivam [8], [40] described the collection of Sinhala-English code-mixed social-media data and proposed machine-learning models for word-level language identification in Latin-script text. They noted that language detection is trivial when Sinhala is written in its own script but ambiguous when it is romanised. More recently, Perera et al. [9] combined a dictionary of known Singlish words, rule-based conversion and a BERT language model to transliterate romanised Sinhala back into Sinhala script.

Transliteration could in principle be used as a pre-processing step that converts Singlish into Sinhala script before classification. This would introduce an additional source of error and would still require a classifier that handles Sinhala script well. It was therefore deferred to future work (Section 9.4).

2.3.3 Sinhala pretrained language models

Dhananjaya et al. [41] carried out the first comprehensive comparison of pretrained language models for Sinhala text classification. They found that XLM-R performed best among multilingual models that include Sinhala, ahead of LaBSE [42] and LASER, and released monolingual Sinhala RoBERTa models. These results concern Sinhala script. They suggest that XLM-R-based models are a reasonable starting point for Sinhala-related text, but they do not measure performance on romanised Sinhala.

2.3.4 Sinhala intent classification and chatbots

Several Sri Lankan studies build intent classifiers for chatbots:

- Fernando and Ganegoda [10] proposed a hybrid intent classification mechanism for a bilingual Sinhala-English restaurant-booking chatbot.
- Avishka et al. [11] applied natural language understanding techniques to intent identification and entity extraction in a Sinhala chatbot.
- Karunanayake et al. [12] classified free-form spoken commands in Sinhala and Tamil into a fixed set of intents using transfer learning from an English speech recogniser.

These works show that intent classification for Sinhala is feasible. However, they deal with Sinhala script or speech, small intent sets and conversational domains such as restaurant booking. None targets free-text household problem descriptions in romanised Sinhala-English, or a taxonomy as fine-grained as 39 trades.

2.3.5 Code-mixed intent detection in other languages

For Indian languages, Banerjee et al. [15] created code-mixed versions of a restaurant-reservation dialogue dataset in Hindi-English, Bengali-English, Gujarati-English and Tamil-English. Khanuja et al. [16] introduced the GLUECoS benchmark for Hindi-English and Spanish-English. They found that multilingual models generally outperform cross-lingual embeddings, and that fine-tuning on artificially generated code-switched data helps further.

These results support the choice of a multilingual encoder for code-mixed text. They also show that the language pairs studied so far do not include Sinhala-English.

2.3.6 Service-request and ticket classification

Automatically routing a free-text request to the right resolver group is a close analogue of mapping a household problem to a worker category:

- Paramesh et al. [13] classified 10,742 IT service-desk tickets into 18 resolution groups using TF-IDF features and ensemble classifiers. A single SVM was among the strongest models (88.23% accuracy).
- Marcuzzo et al. [14] compared BERT-based and SVM classifiers on short, noisy software support tickets. They found that the choice of embedding strategy and the use of hierarchical label relationships both noticeably affect accuracy.

These studies use English, domain-specific vocabulary and labels drawn from real operational data. Romanised code-mixed text and consumer household language are not addressed.

2.3.7 Synthetic training data for intent classification

Because labelled data is scarce, researchers increasingly generate training utterances with large language models. Sahu et al. [43] showed that prompting an off-the-shelf language model to generate intent examples improves classifiers when intents are clearly distinct. When intents are semantically close, the model "often generates utterances that belong to a closely-related intent instead of the desired one", which reduces label fidelity. This finding is directly relevant here: the dataset for Component 01 was largely AI-generated, and several of its categories are close neighbours (Section 6.1.4). Simpler augmentation methods, such as synonym replacement and random insertion [44], are an alternative but do not create code-mixed text.

2.4 Existing Systems, Products and Techniques

Table 2.1 summarises how existing service platforms help a customer find a worker. All of them rely on the customer choosing a category or typing a keyword that matches listing text. TaskRabbit's support documentation explicitly advises customers to choose the closest category when a task "does not fall neatly into one category" [17]. Local platforms such as ikman.lk organise worker advertisements under category and location filters [18]. Work.lk, a Sri Lankan freelance marketplace, was reported in

2019 to have more than 20,000 registered skilled professionals, including plumbers and electricians [19]. HodaBass.lk, a Sri Lankan app by Xiteb, lets tradespeople such as carpenters, plumbers, electricians and painters advertise their services free of charge, and customers find them by trade category [20]. None of these platforms offers free-text understanding of romanised Sinhala-English problem descriptions.

Table 2.1: Comparison of existing worker-finding platforms with the proposed component

Platform	**Region**	**How the customer finds a worker**	**Free-text problem understanding**	**Romanised Singlish support**
TaskRabbit [17]	Global (not Sri Lanka)	Choose task category, then describe task	No - category chosen by customer	No
ikman.lk [18]	Sri Lanka	Browse category and location; keyword search over advertisements	No - keyword match	No
work.lk [19]	Sri Lanka	Browse or search professional categories	No - category and keyword	No
HodaBass.lk [20]	Sri Lanka	Browse worker advertisements by trade category	No - category chosen by customer	No
Facebook community groups	Sri Lanka	Post a problem and wait for replies	Human readers only	Human readers only
Proposed Component 01	Sri Lanka	Type the problem in own words	Yes - 39-category classifier	Yes - English and romanised Singlish

Techniques that could implement free-text matching fall into three families. Table 2.2 compares them against the needs of this project.

- **Lexical methods** (keyword rules, TF-IDF with linear classifiers) are simple and fast. They depend on consistent spelling, which romanised Sinhala does not have.
- **Sub-word and character models**, such as FastText [45] or character n-grams, are robust to spelling variation but need enough in-domain text to learn useful representations.
- **Pretrained transformer encoders**, either fine-tuned end to end [28], [29] or used as fixed feature extractors with a light classifier [26], [46], offer semantic generalisation. They differ greatly in computational cost.

Table 2.2: Candidate technique families for Component 01

Technique family	Strengths	Weaknesses for this project
Keyword / TF-IDF + linear model	Very fast, interpretable	Breaks on unstandardized Singlish spelling; no semantics
Character n-grams / FastText	Robust to misspellings	Needs larger in-domain corpus; limited semantics
Fine-tuned transformer (mBERT, XLM-R, SinBERT)	Highest potential accuracy	GPU training, larger model, more effort to serve; SinBERT targets Sinhala script
Fixed sentence encoder + nearest neighbour (FAISS)	No training; easy to update by adding examples	Sensitive to individual noisy neighbours
Fixed sentence encoder + SVM (selected)	Trains on CPU in minutes; probability outputs; learns boundaries using all training data	Encoder not trained on Sinhala; quadratic SVM training cost as data grows

2.5 Gap Synthesis and Conceptual Framework

Bringing these strands together, the research gap is as follows. Sri Lankan household-service platforms require users to choose a service category before describing their problem. Research on Sinhala-English code-mixed text has focused on social-media classification (sentiment, hate speech, humour, aspects), language identification and transliteration [6], [8], [9]. Existing Sinhala intent classifiers target Sinhala-script or bilingual chatbots with small intent sets in domains such as restaurants and spoken commands [10], [11], [12]. Service-request classification has been studied for English help-desk tickets [13], [14], and code-mixed intent detection mainly for Hindi-English [15], [16]. To the best of the author's knowledge, no published work classifies free-text household problem descriptions written in romanised Sinhala-English into fine-grained worker categories (39 trades), and no such dataset exists.

Figure 2.1 presents the conceptual framework that follows from this gap. Existing research and products supply the building blocks: multilingual sentence encoders, SVM classification with probability outputs, confidence-based rejection, and evidence on synthetic data. This thesis combines them into a working component that is evaluated on a new 39-category dataset and integrated into a marketplace.

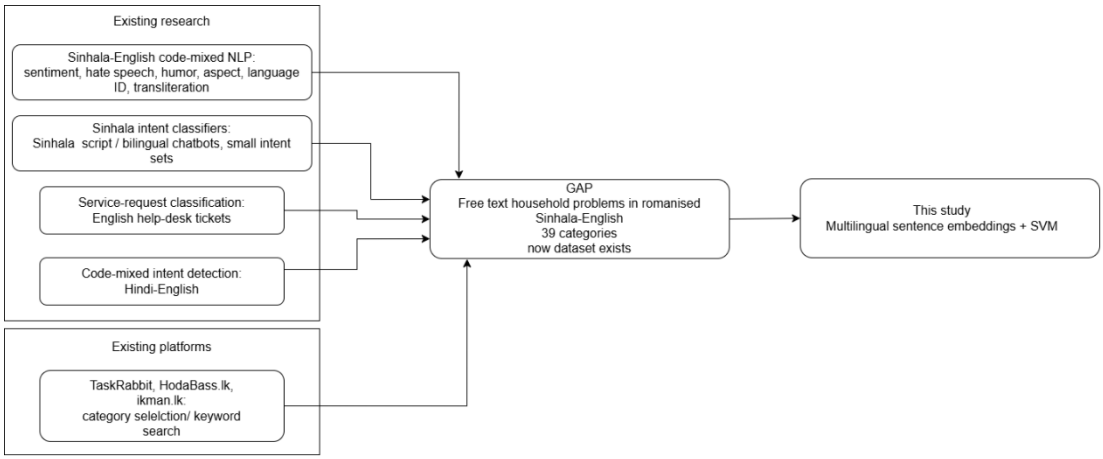


Figure 2.1: Conceptual framework: positioning of this study relative to existing research and platforms

3 RESEARCH METHODOLOGY

3.1 Research Design

The research follows a design-science approach with a quantitative experimental evaluation. The artefact, an intent recognition component, is designed to solve a practical problem identified in Chapter 1. It is then evaluated through controlled experiments on a held-out test set. Figure 3.1 summarises the workflow.

The experimental part is a comparative study of two ways of turning the same sentence embeddings into a category prediction:

- **Approach 1 - similarity search.** Each training sentence is encoded and stored in a FAISS inner-product index. A query is assigned the label of its single most similar training sentence (cosine top-1 nearest neighbour).
- **Approach 2 - trained classifier.** An RBF-kernel SVM is trained on the training embeddings and predicts the label of a query embedding directly, together with class probabilities.

Both approaches use the same dataset file, the same stratified split with the same random seed, and the same embedding model with the same settings. Any difference in results can therefore be attributed to the decision mechanism rather than to the data or the features.

After the comparison, an error analysis led to a second, separate experiment on a cleaned version of the dataset (Experiment B, Section 3.3.5). The two experiments use different data and are reported separately.

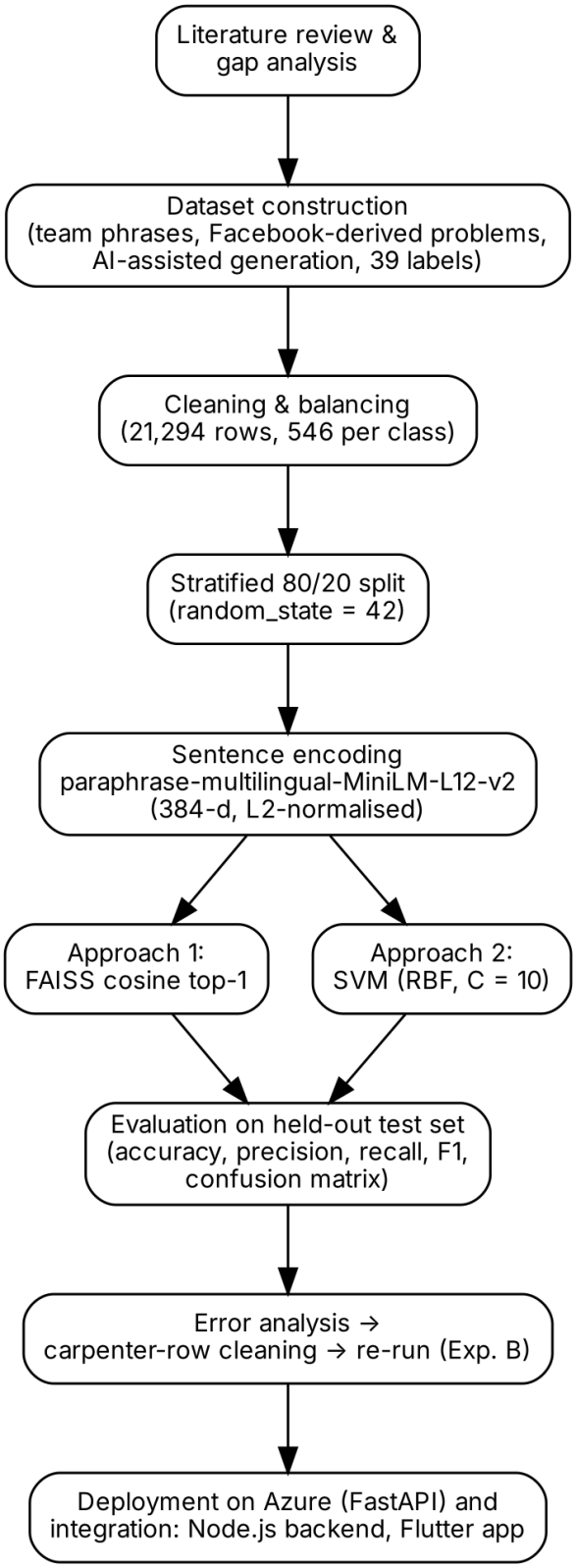


Figure 3.1: Research workflow followed for Component 01

3.2 Data, Participants and Study Materials

3.2.1 Category taxonomy

Earlier drafts of the component predicted a broad worker category together with a specific issue type. This was simplified: sub-types that customers need to distinguish were promoted to categories of their own. For example, a house cleaner and a washroom cleaner, or a bathroom water heater and a kitchen water heater technician, became separate categories. The issue-type field was dropped.

The final taxonomy has 39 worker categories, which are listed in Appendix A, Table A.1. Labels are stored in lower case with underscores, for example ac_technician and water_pump_technician.

3.2.2 Data sources and construction

No public dataset of Sri Lankan household problem descriptions exists, so a new dataset was constructed. It has two columns: problem_description (free text) and worker_type (one of the 39 labels). The rows come from three sources:

1. **Team-written seed phrases.** Group members wrote short, realistic descriptions in English and romanised Singlish for many of the categories, based on their own experience of household problems.
2. **Problems derived from public social media.** Problems that people had posted, and comments made in reply, in public Sri Lankan Facebook groups were read. New problem descriptions were written based on them. Posts were not copied verbatim, and no names, profile information or other personal data were stored.
3. **AI-assisted generation.** Using the seed phrases and the Facebook-derived problems as examples, an AI assistant (Claude) was prompted to generate further descriptions for each target category in both English and romanised Singlish, with varied phrasing, spelling and level of detail. Most rows in the dataset come from this step.

Because each row was produced for a specific target category, its label was assigned at generation time. There was no independent human annotation and no inter-annotator agreement check. Rows that were obviously wrong, duplicated or ambiguous were

removed or corrected manually, and labels were normalised to lower case with underscores. The full use of AI in this work is disclosed in Appendix B.

3.2.3 Dataset profile

The dataset used for the main comparison (dataset_balanced.xlsx, sheet "All Data") contains 21,294 rows. Every one of the 39 categories has exactly 546 rows. Table 3.1 and Figure 3.2 summarise its properties.

To estimate how much of the data is code-mixed, a simple keyword heuristic was applied. It marked a row as romanised Singlish if it contained at least one common Sinhala function word or verb form (for example eka, eke, wada, karanna, oona, thiyenawa, nisa or wela). By this heuristic, 8,877 rows (41.7%) are romanised Singlish and 12,417 rows (58.3%) are English. The Singlish share per category ranges from 29% to 57%. The heuristic is approximate: short Singlish rows without any listed word are counted as English.

Descriptions are short, with a mean of 9.6 words (median 10, range 2 to 20) and 60.6 characters. The English rows show two peaks: very short keyword-style queries of three to four words (for example "concrete sump tank cleaning") and full sentences of 10 to 14 words. Five exact duplicate descriptions remain in the file.

Table 3.1: Profile of the balanced dataset used in Experiment A

Property	Value
Total rows	21,294
Worker categories	39
Rows per category	546 (perfectly balanced)
Columns	problem_description, worker_type
Romanised Singlish rows (heuristic)	8,877 (41.7%)
English rows (heuristic)	12,417 (58.3%)
Mean / median length	9.6 / 10 words; 60.6 characters
Shortest / longest description	2 / 20 words
Exact duplicate descriptions	5

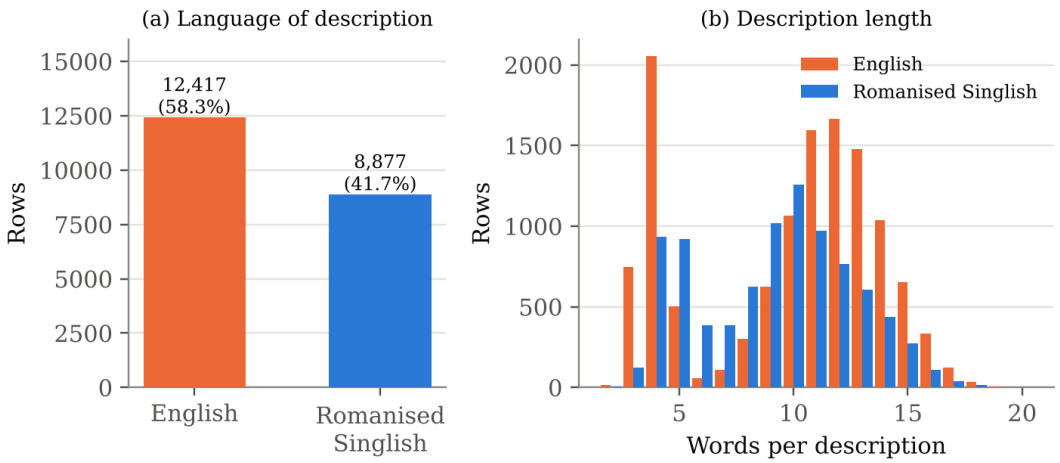


Figure 3.2: Dataset composition: (a) estimated language of each description, (b) distribution of description length by language

Table 3.2 shows representative rows, taken from a random sample of the dataset, to illustrate the mix of English and romanised Singlish, technical and everyday vocabulary, and short and long descriptions.

Table 3.2: Representative rows from the dataset

Problem description	Worker type
Pet animals lage ganda AC eken enawa.	ac_technician
The pipe going to the rainwater harvesting tank is blocked with mud.	drain_unblocking
Sofa eke legs tika lii walin liyawala usa wadi karanna.	furniture_restoration
lin pump eka clean karanna	well_cleaner
Gaha udin yana bat/bird nests nisa gasa height eka bagekin adu karanna.	tree_service_specialist
concrete sump tank cleaning	water_tank_cleaner
Bima kema watila kurumbiyo awith lokuwata mop karanna oni.	house_cleaner
solar water heater not working	bathroom_water_heater
Aluth bathroom ekata beveled edge kannadiyak danna oni.	glass_installer
Wassa dawas nisa biththi wala paint eka bubu danna aragena.	painter
Kitchen eke chimney eka eliyata danna bittiyen hilak kapanna.	mason

3.2.4 Cleaned dataset for Experiment B

Error analysis of Experiment A (Section 6.1.4) showed that the carpenter category was the weakest. Many of its rows overlapped with furniture restoration, ceiling installation and roof repair. Ambiguous carpenter rows were therefore removed and all categories

were rebalanced to the same size. This produced a cleaned dataset of about 19,400 rows, with about 498 rows per category, which was used for Experiment B.

3.3 Experimental Procedure

3.3.1 Loading and splitting

Both notebooks follow the same steps:

1. Load the "All Data" sheet and strip whitespace from the column names.
2. Keep the problem_description and worker_type columns and drop rows with missing values.
3. Split the data with scikit-learn's train_test_split [35], using test_size = 0.20, stratify = worker_type and random_state = 42.

For Experiment A this gives 17,035 training rows (436 or 437 per category) and 4,259 test rows (109 or 110 per category). The text itself is not pre-processed: there is no lower-casing, punctuation removal, stop-word removal or spelling normalisation. The raw description is passed directly to the embedding model, whose own tokenizer handles casing and sub-word segmentation.

3.3.2 Sentence encoding

All descriptions are encoded with paraphrase-multilingual-MiniLM-L12-v2 through the sentence-transformers library [30]. The settings are batch_size = 64 and normalize_embeddings = True. Internally, the model:

1. Tokenises the text into SentencePiece sub-word pieces [32].
2. Passes them through 12 transformer layers.
3. Mean-pools the token vectors into one 384-dimensional sentence vector.

L2 normalisation then scales each vector to unit length, so that the inner product of two vectors equals their cosine similarity. The model is used exactly as published, without fine-tuning.

3.3.3 Approach 1: FAISS similarity search

The 17,035 training vectors are added to a FAISS IndexFlatIP index (exact inner-product search) [39]. For each test vector, the index returns the training vector with the highest cosine similarity, and that vector's label is taken as the prediction (k = 1).

3.3.4 Approach 2: SVM classifier

Labels are converted to integers with a LabelEncoder. An SVC classifier [35], [34] is trained on the training embeddings with the settings in Table 3.3. The RBF kernel lets the classifier learn non-linear boundaries between categories in the embedding space. C = 10 applies a relatively weak regularisation, allowing the model to fit the training data closely. gamma = "scale" sets the kernel width from the variance of the features. class_weight = "balanced" reweights classes inversely to their frequency; it has no effect on this perfectly balanced dataset but protects against imbalance in future versions. probability = True enables Platt-scaled probability estimates [36], [37], which are used for the confidence score and the rejection rule.

Table 3.3: Configuration of the final model

Setting	Value
Embedding model	sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2 (pretrained, not fine-tuned)
Tokenizer	XLM-R SentencePiece (sub-word)
Embedding size	384 dimensions, mean pooling, L2-normalised
Text pre-processing	None (raw text)
Classifier	scikit-learn SVC
Kernel	RBF
C	10
gamma	"scale"
class_weight	"balanced"
probability	True (Platt scaling with pairwise coupling)
random_state	42
Train / test split	Stratified 80% / 20%, random_state = 42
Rejection threshold	Maximum class probability < 0.4 -- "not found"

3.3.5 Experiment B: cleaned dataset

After the error analysis, the same SVM configuration was trained and evaluated on the cleaned dataset described in Section 3.2.4, again with a stratified 80/20 split and random_state = 42. The test set contained 3,885 rows (99 to 100 per category). Because the data differs from Experiment A, the results of Experiment B are not compared directly with the Experiment A figures.

3.4 Measures and Evaluation Metrics

Performance is measured on the held-out test set with the following metrics [47]. For a category c, let TP_c, FP_c and FN_c be the numbers of true positives, false positives and false negatives.

$$Accuracy = correctly\ classified\ descriptions\ /\ all\ test\ descriptions \quad (3.1)$$

$$Precision_c = TP_c / (TP_c + FP_c), \quad Recall_c = TP_c / (TP_c + FN_c) \quad (3.2)$$

$$F1_c = 2 \times Precision_c \times Recall_c / (Precision_c + Recall_c) \quad (3.3)$$

$$Macro\text{-}F1 = (1 / 39) \times \Sigma_c\ F1_c \quad (3.4)$$

Precision answers the question "when the model says plumber, how often is it right?". Recall answers "of all real plumbing problems, how many did the model find?". The macro average treats every category equally, which suits a marketplace in which every trade matters. The weighted average weights each category by its number of test samples; because the test set is balanced, it is almost identical to the macro average.

A row-normalised confusion matrix is used to see which categories are mistaken for which. A 95% confidence interval for accuracy is computed with the normal approximation, accuracy ± 1.96 × √(accuracy × (1 − accuracy) / n). For reference, a classifier that guesses at random among 39 balanced categories would reach about 2.6% accuracy.

3.5 Data Analysis

The analysis proceeds in four steps:

1. Overall accuracy and macro-averaged precision, recall and F1 are compared between the two approaches (RQ2).

2. Per-category F1-scores are compared to see where the SVM gains or loses relative to FAISS.
3. The SVM confusion matrix and the most frequent off-diagonal cells are examined to identify systematically confused category pairs (RQ3).
4. A set of short, realistic Singlish and English queries is passed through the trained model to inspect its confidence scores qualitatively and to see how the 0.4 threshold would act.

3.6 Ethics, Privacy, Security and Responsible AI

Personal data. The dataset contains no personal data. Problems found in public Facebook groups were used only as inspiration for newly written descriptions. No names, profile links, images, locations or verbatim posts were stored. At run time, the component receives only the problem text typed by the customer. It does not need the customer's identity, and it does not store the query beyond what the backend logs for operating the service. Sri Lanka's Personal Data Protection Act, No. 9 of 2022 [48], is being brought into operation in stages. The marketplace's handling of customer accounts and logs should follow its principles of purpose limitation, data minimisation and security. The component's own design already minimises the data it receives.

Synthetic data and transparency. The use of AI-generated data is disclosed in this thesis (Section 3.2.2 and Appendix B). Its effect on the validity of the results is discussed in Section 7.5. Following the practice of datasheets for datasets [49] and model cards [50], the dataset's sources, labelling process and known limitations, and the model's intended use and evaluation conditions, are documented in this thesis.

Responsible behaviour. A wrong category sends a customer to the wrong worker, which wastes time but is not dangerous in itself. The 0.4 confidence threshold makes the component decline to answer when it is unsure, instead of confidently returning an arbitrary trade. The component does not rank or judge workers; that is the role of Components 02 and 03.

Security. The service accepts free text only, has no access to the database, and returns one label and one number. In the current Azure deployment the endpoint is reachable

over HTTPS; restricting it to the backend, for example with an API key or network access rules, is recommended (Section 5.5).

3.7 Reproducibility and Research Artefacts

The experiments were run in Google Colab notebooks (Component01_FAISS_Pipeline.ipynb and Component01_SVM_Pipeline.ipynb), using the sentence-transformers, faiss-cpu, scikit-learn, pandas, openpyxl, matplotlib and seaborn libraries. All random processes use random_state = 42.

The following artefacts are kept by the group and can be provided to examiners on request:

- The dataset file, dataset_balanced.xlsx.
- The two notebooks, with their saved outputs.
- The trained model files (svm_model.pkl and label_encoder.pkl), which are also stored in the private Hugging Face model repository used by the deployed service.
- The per-sample test results (svm_evaluation_results.csv).

Because the embedding model is downloaded by name from the Hugging Face hub, the exact model version should be pinned (for example by recording its commit hash) in any future re-run.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The requirements were derived from the problem statement, the group architecture and the needs of Component 02, which consumes this component's output. Table 4.1 lists the functional requirements and Table 4.2 the non-functional requirements. Non-functional requirements are grouped using the quality characteristics of ISO/IEC 25010 [51].

Table 4.1: Functional requirements of Component 01

ID	Requirement
FR1	Accept one free-text problem description from the mobile application, in English or romanised Singlish.
FR2	Predict exactly one worker category from the 39-category taxonomy.
FR3	Return a confidence score (probability) for the predicted category.
FR4	If the highest probability is below 0.4, return a "not found" result instead of a category, so the application can ask the user to try again.
FR5	Return the result as JSON with the field worker_type, whose value matches the category labels used by Component 02.
FR6	Load the trained model once at service start-up and reuse it for every request.

Table 4.2: Non-functional requirements of Component 01

ID	Quality characteristic	Requirement
NFR1	Functional suitability	Macro-F1 and accuracy on the held-out test set should clearly exceed the similarity-search baseline.
NFR2	Performance efficiency	Inference should run on a CPU without a GPU; the model should be small enough for a standard cloud instance.
NFR3	Compatibility	Expose a language-neutral HTTP/JSON interface that the Node.js backend can call.
NFR4	Usability	Customers must not have to know trade names or choose from a list.
NFR5	Reliability	Malformed or empty requests must return an error, not crash the service.
NFR6	Security	The service is called only by the backend, receives no personal data, and has no database access.
NFR7	Maintainability	New categories or examples can be added by re-running the training notebook; model artefacts are versioned files.

4.2 Component Architecture within the Complete Solution

Figure 1.1 showed the complete solution. Component 01 sits between the customer and the rest of the backend. The Flutter application sends the customer's text to the Node.js backend. The backend calls the Component 01 service, receives the predicted worker category, and passes that category to Component 02, which ranks suitable workers. Component 03 runs independently: it updates trust scores in the database whenever a review is submitted, and Component 02 reads those scores. Component 04, the planned fair-price predictor, will be called by the backend once a worker category and job are known.

The integration contract between Components 01 and 02 is deliberately narrow. Component 01 outputs one field, worker_type, which corresponds to the "Worker Category" input of Component 02. Component 02's separate "Worker Type" filter (for example individual versus company) is not produced by Component 01. Keeping the contract to a single, well-defined label means that either component can be changed without affecting the other, provided the 39 category labels stay the same.

Figure 4.1 shows the internal architecture of the Component 01 service. It runs as an independent Python microservice built with FastAPI. At start-up it loads three artefacts: the embedding model, the trained SVM and the label encoder. Each request then passes through four stages: validation, encoding, probability estimation and the decision rule.

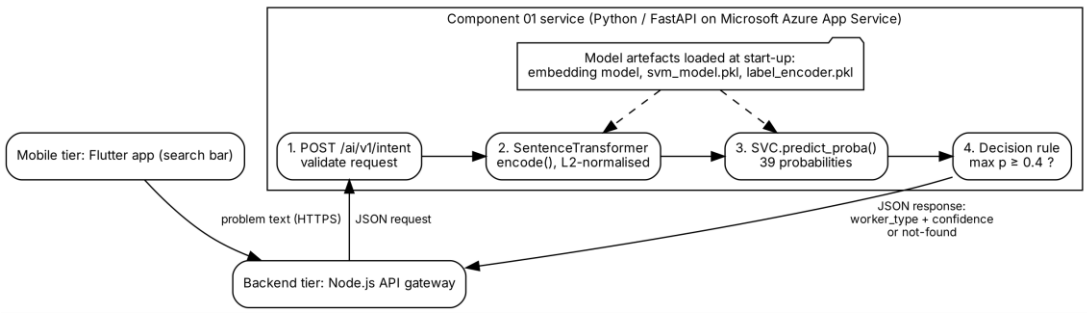


Figure 4.1: Internal architecture of the Component 01 service and its connection to the backend and mobile tiers

4.3 Design Alternatives and Decisions

Table 4.3 records the main design decisions, the alternatives that were considered and the reasons for each choice.

Table 4.3: Design decisions for Component 01

Decision	Alternatives considered	Choice and rationale
Input method	Category menu; keyword search; free text	Free text - removes the need for the customer to know trade names (NFR4).
Language handling	Transliterate Singlish to Sinhala script first; spelling normalisation dictionary; rely on sub-word tokenizer	Rely on the multilingual model's sub-word tokenizer - no hand-built Singlish resources exist; transliteration adds a second error source. Spell correction deferred (Section 9.4).
Text representation	TF-IDF; character n-grams; FastText; fine-tuned transformer; pretrained sentence embeddings	Pretrained multilingual sentence embeddings - semantic features without GPU training; compact 384-d vectors.
Embedding model	LaBSE; larger multilingual MPNet; paraphrase-multilingual-MiniLM-L12-v2	MiniLM-L12 - small and fast enough for CPU serving, while trained on paraphrase data in about fifty languages.
Decision mechanism	FAISS nearest neighbour; k-NN voting; SVM	SVM (RBF) - highest accuracy in the controlled comparison (Section 6.1) and provides calibrated probabilities.
Output granularity	Category + issue type (two models); flat category	Single flat label over 39 categories – sub-types that matter were promoted to categories; simpler contract for Component 02.
Low-confidence behaviour	Always return top-1; show top-3 suggestions; ask a clarifying question; reject below threshold	Reject below 0.4 - simple and safe; avoids sending users to an unrelated trade. Top-3 suggestions noted as an enhancement.
Service interface	Embed the model in the Node.js process; Python microservice	Python FastAPI microservice - the ML stack is in Python; independent scaling and deployment.

An earlier version of the comparison also tested k = 10 similarity-weighted voting over FAISS neighbours. That test was run on an older, unbalanced version of the dataset, so its result is not comparable with the experiments reported here and is not used as evidence in this thesis.

4.4 Detailed Design

4.4.1 Inference flow

Figure 4.2 shows the processing of a single request:

1. The text is validated: it must be present and non-empty.
2. The embedding model tokenises it into sub-word pieces and produces one L2-normalised 384-dimensional vector.
3. The SVM computes a probability for each of the 39 categories.
4. If the largest probability is at least 0.4, the corresponding label is decoded with the label encoder and returned with its probability. Otherwise, a "not found" result is returned and the application shows the message "We couldn't find the worker, try again."

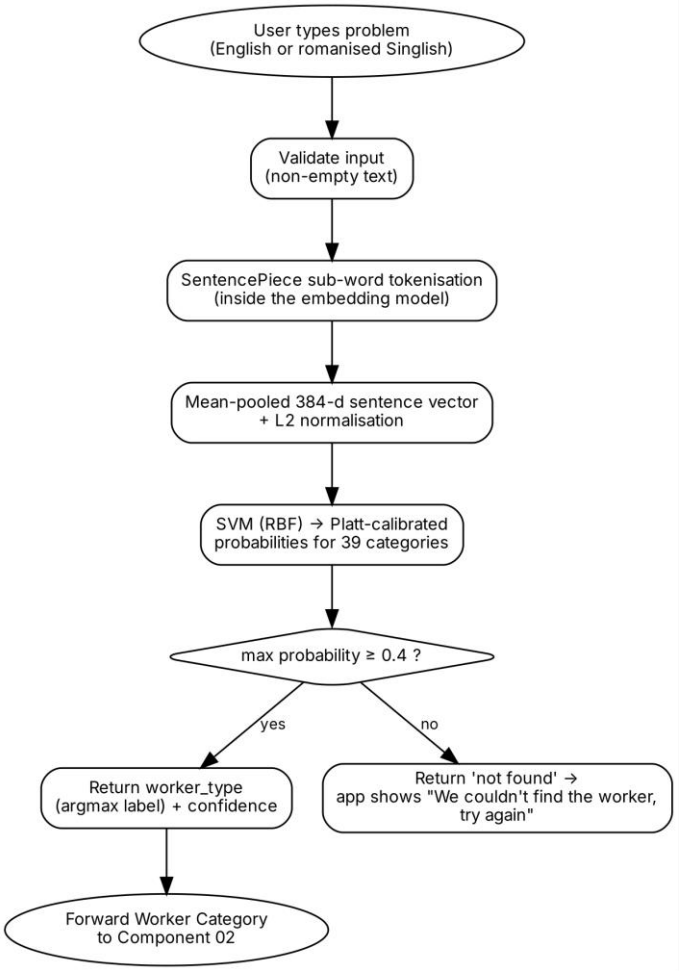


Figure 4.2: Inference flow of Component 01, including the 0.4 confidence rule

4.4.2 Decision rule

Let p_c(x) be the probability that the SVM assigns to category c for the description x. The prediction and the rejection rule are:

$$\hat{c} = \text{argmax}_c \, p_c(x), \quad \text{confidence} = \max_c p_c(x) \tag{4.1}$$

$$\text{output} = \hat{c} \ \text{ if confidence} \geq 0.4; \quad \text{output} = \text{"not found"} \ \text{ otherwise} \tag{4.2}$$

With 39 categories, a uniform guess would give each category a probability of about 0.026. A threshold of 0.4 therefore requires the model to place a clear plurality of its probability mass on one category. In the notebook's sample queries (Section 6.2.2), clear requests such as "AC eka sisil wenne na" received probabilities above 0.85. An ambiguous request, "lin sudda karanawa" (cleaning the well), received only 0.17 for its top category, so the threshold would correctly reject it rather than return the wrong trade. The value 0.4 was chosen by the team as a practical default. It has not yet been tuned against measured coverage and error rates; Section 9.4 proposes how to do this.

4.4.3 Interface contract

The service exposes a single prediction endpoint that accepts and returns JSON. Figure 4.3 shows the request and response formats as designed. In the deployed service the endpoint is exposed as POST /ai/v1/intent, and a root endpoint reports the service status and the version of the loaded model.

```
Request (from Node.js backend):
{ "problem_description": "watura bata pupurala" }

Response – category found:
{ "worker_type": "plumber", "confidence": 0.894 }

Response – below threshold:
{ "worker_type": null, "confidence": 0.174, "message": "We couldn't
find the worker, try again." }
```

Figure 4.3: Request and response formats of the Component 01 prediction endpoint

The value of worker_type is one of the 39 lower-case labels (Appendix A, Table A.1). Component 02 uses it directly as its Worker Category filter.

4.4.4 Module structure

The service code is organised into four parts:

1. A start-up routine that loads the sentence-transformer model, svm_model.pkl and label_encoder.pkl into memory.
2. Request and response schemas that validate the input text and define the output fields.

3. A predict function that encodes the text with normalize_embeddings = True, calls predict_proba, applies the decision rule and decodes the label.
4. The FastAPI route that connects the HTTP request to the predict function.

The training code stays in the Colab notebook. It is kept separate from the serving code so that the model can be retrained and the service simply restarted with new artefacts.

4.5 Standards and Professional Practice

The design and documentation of the component follow these standards and practices:

- **Software product quality.** The non-functional requirements and the test plan are framed using the quality characteristics of ISO/IEC 25010 [51]: functional suitability, performance efficiency, compatibility, usability, reliability, security and maintainability.
- **Software testing.** Test design follows the concepts of ISO/IEC/IEEE 29119 [52]: test items, test conditions, test cases with expected results, and a record of actual results.
- **AI risk management.** Risks such as data bias, synthetic-data artefacts and over-confident predictions are identified and mitigated in line with the NIST AI Risk Management Framework [53]. Mitigations include disclosure, held-out evaluation and a rejection threshold.
- **Data protection.** Data handling is designed to be consistent with Sri Lanka's Personal Data Protection Act, No. 9 of 2022 [48].
- **Documentation.** The dataset and model are documented using the datasheet and model-card practices [49], [50].
- **Engineering practice.** Model artefacts are kept as versioned files, every experiment uses a fixed random seed, and training code is separated from serving code.

5 IMPLEMENTATION, INTEGRATION AND TESTING

5.1 Development Environment and Technology Stack

Table 5.1 lists the tools and libraries used for each part of the component.

Table 5.1: Technology stack of Component 01

Layer	Technology	Purpose
Experimentation	Google Colab (Python notebooks), Google Drive	Data loading, training and evaluation
Data handling	pandas, openpyxl	Reading the Excel dataset, splitting, reporting
Embeddings	sentence-transformers; paraphrase-multilingual-MiniLM-L12-v2	Sub-word tokenisation and 384-d sentence vectors
Similarity search (baseline)	faiss-cpu (IndexFlatIP)	Exact cosine nearest-neighbour search
Classifier	scikit-learn SVC, LabelEncoder, metrics	SVM training, probabilities and evaluation
Visualisation	matplotlib, seaborn	Per-category charts, confusion matrices
Model persistence	pickle	svm_model.pkl, label_encoder.pkl
Serving	Python, FastAPI [2]	HTTP/JSON prediction endpoint
Backend	Node.js	Calls Component 01, then Component 02
Mobile client	Flutter [1]	Search bar and result screens

5.2 Component Implementation

5.2.1 Data loading and splitting

The dataset is read from Google Drive with pandas. Column names are stripped of whitespace and rows with missing values are dropped. The stratified 80/20 split uses random_state = 42. The notebook prints the resulting sizes: 17,035 training rows and 4,259 test rows, with at least 436 and 109 rows per category respectively.

5.2.2 Encoding

The embedding model is loaded once by name from the Hugging Face hub, and its embedding dimension (384) is checked. Training and test sentences are encoded in batches of 64 with normalize_embeddings = True and returned as NumPy arrays.

In the Colab session, encoding the 17,035 training sentences took 9.0 seconds and encoding the 4,259 test sentences took 1.8 seconds. That is roughly 0.4 to 0.5 milliseconds per sentence when processed in batches. The type of Colab hardware was not recorded, and batch throughput is not the same as single-request latency, so these figures indicate only that encoding is not a bottleneck. They are not used as a response-time measurement.

5.2.3 FAISS baseline

The FAISS notebook builds an IndexFlatIP over the 17,035 training vectors, which take about 25 MB of memory. It then searches the index with the test vectors using k = 1 and maps each returned index back to its training label.

5.2.4 SVM training

The SVM notebook encodes labels with a LabelEncoder (39 classes, indices 0 to 38, in alphabetical order). It trains an SVC with the configuration in Table 3.3. Training took 203.6 seconds on the Colab CPU.

The fitted model keeps 14,120 support vectors, which is 82.9% of the training set. Such a high proportion indicates that the categories overlap considerably in the embedding space: most training points lie on or inside the margin of at least one pairwise boundary. This is consistent with the confusions analysed in Section 6.1.4.

After evaluation, the notebook saves:

- the classifier (svm_model.pkl);
- the label encoder (label_encoder.pkl);
- a CSV file with every test sentence, its actual label, its predicted label and whether the prediction was correct (svm_evaluation_results.csv);
- the per-category chart and the confusion matrix.

5.2.5 Inference function

The notebook's inference function, which the service reuses, works in five steps:

1. Encode the query with normalize_embeddings = True.
2. Call predict_proba.

3. Sort the 39 probabilities.
4. Decode the top label.
5. Return it with its confidence.

In the service, the 0.4 rule (Equation 4.2) is applied to the top probability before the result is returned.

5.3 Integration with the Other Components

Integration was carried out together with the rest of the group, as shown in Figure 4.1. The Component 01 service runs on Microsoft Azure, and the Node.js backend calls it over HTTPS:

1. The Flutter application sends the customer's problem text to the Node.js backend through its search endpoint (POST /api/v1/search).
2. The backend forwards the text to the Component 01 FastAPI service on Azure as JSON (POST /ai/v1/intent).
3. If the service returns a worker_type, the backend uses it as the Worker Category filter for Component 02. Component 02 removes workers in other categories and offline workers, scores the remaining workers, and returns the top twenty. The application then displays them.
4. If the service returns "not found", the application displays "We couldn't find the worker, try again."

Component 03 is not called during a search. Its trust scores are already stored per worker and per category, and Component 02 reads them from the database.

Integration required agreement on one point: the category labels. Component 02's worker records and Component 03's category-specific trust scores must use exactly the same 39 lower-case labels that Component 01 produces. Otherwise a correct prediction would match no workers. The label list in Appendix A, Table A.1 is the shared reference.

5.4 Verification and Software Testing

Testing of Component 01 has three levels:

1. Model-level evaluation on the held-out test set. This is the primary evidence and is reported in Chapter 6.
2. Functional tests of the inference function and the service.
3. Integration tests through the backend and the application.

Table 5.2 lists the test cases and their results. The results of T1–T7 come from the saved notebook outputs; T8–T11 were obtained by testing the deployed service and the integrated application.

Table 5.2: Test cases for Component 01

ID	Level	Test condition / input	Expected result	Result
T1	Model	Held-out test set, Experiment A (4,259 rows)	SVM accuracy and macro-F1 above FAISS baseline	Observed: SVM 78.68% vs FAISS 71.50%; macro-F1 0.79 vs 0.71- Pass
T2	Model	Held-out test set, Experiment B (3,885 rows)	Comparable performance after cleaning	Observed: 78.35% accuracy; macro-F1 0.78 - Pass
T3	Function	"watura bata pupurala"	plumber, confidence ≥ 0.4	Observed: plumber, 0.894 - Pass
T4	Function	"current eka wada na"	electrician, confidence ≥ 0.4	Observed: electrician, 0.963 - Pass
T5	Function	"AC eka sisil wenne na"	ac_technician	Observed: ac_technician, 0.9998 - Pass
T6	Function	"kussiye sink eke upper instant heater"	kitchen_water_heater	Observed: kitchen_water_heater, 0.971 - Pass
T7	Function	"lin sudda karanawa" (ambiguous)	Low confidence - "not found"	Observed: top probability 0.174 (washroom_cleaner) - rejected by rule - Pass
T8	Service	Empty string or missing field	Validation error response, service keeps running	Observed: validation error returned; service kept running - Pass
T9	Service	Very long text (> 128 tokens)	Prediction returned; text truncated by model	Observed: prediction returned without error - Pass
T10	Integration	Query sent from Flutter search bar	Worker list from Component 02 for the predicted category	Observed: correct list of workers returned - Pass
T11	Integration	Unrelated query (e.g. "I want a pizza")	"We couldn't find the worker, try again."	Observed: “try again” message displayed - Pass

5.5 Deployment and Operation

The Component 01 service is deployed on Microsoft Azure App Service (Linux, Basic B1 plan, Central India region), where the FastAPI application runs under Uvicorn. The deployment works as follows:

1. The trained SVM, the label encoder and a small metadata file (holding the model version) are stored in a private Hugging Face model repository.
2. At start-up, the service downloads these artefacts together with the pretrained embedding model and loads them into memory once; the start-up time limit was extended to allow for this first download.
3. The "Always On" setting keeps the service loaded, so requests do not wait for the model to be reloaded after a period of inactivity.
4. To release a retrained model, the new svm_model.pkl and label_encoder.pkl are uploaded to the repository, the version number in the metadata file is increased, and the App Service is restarted. The application code and the URL used by the backend stay the same.

For operation, the service should log:

- the predicted category;
- the confidence;
- whether the 0.4 rule rejected the query;

In future, these logs, without user identity, would provide the first real-world data for measuring the rejection rate, discovering missing categories and collecting real Singlish queries for retraining (Section 9.4).

6 RESULTS AND VALIDATION

The results are reported for two separate experiments, which must not be mixed:

- **Experiment A** compares FAISS similarity search with the SVM classifier on the balanced 21,294-row dataset. Both methods use the same 4,259-row test set.
- **Experiment B** evaluates the same SVM configuration after the carpenter rows were cleaned and the data rebalanced, on a different 3,885-row test set.

6.1 Research Results

6.1.1 Experiment A: overall comparison

Table 6.1 and Figure 6.1 summarise the comparison. The SVM classifier correctly categorised 78.68% of the test descriptions, compared with 71.50% for FAISS top-1 search. This is an improvement of 7.18 percentage points, and it means about 306 more of the 4,259 test descriptions were categorised correctly. Macro-averaged precision, recall and F1 rose from 0.72, 0.71 and 0.71 to 0.79, 0.79 and 0.79.

The 95% confidence intervals for accuracy do not overlap: [70.14%, 72.86%] for FAISS and [77.45%, 79.91%] for the SVM. The difference is therefore very unlikely to be due to the particular test sample. Both results are far above the 2.6% that random guessing among 39 balanced categories would achieve.

Table 6.1: Experiment A - overall results on the same 4,259-row test set

Metric	Embeddings + FAISS (top-1)	Embeddings + SVM (RBF)	Difference
Accuracy	71.50%	78.68%	+7.18 pts
95% CI for accuracy	70.14 – 72.86%	77.45 – 79.91%	–
Macro precision	0.72	0.79	+0.07
Macro recall	0.71	0.79	+0.08
Macro F1-score	0.71	0.79	+0.08
Weighted F1-score	0.71	0.79	+0.08
Training time	Index build only	203.6 s (CPU)	–

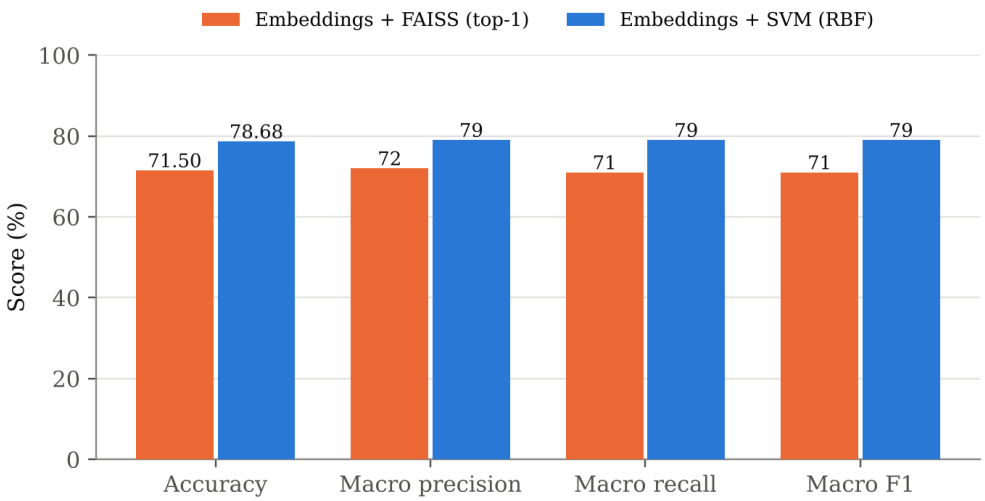


Figure 6.1: Experiment A: accuracy and macro-averaged precision, recall and F1-score of the two approaches

6.1.2 Experiment A: per-category results

Figure 6.2 compares the F1-score of each category under the two approaches; Appendix A, Table A.2 gives the full precision, recall and F1 values. The SVM improved F1 for 36 of the 39 categories, matched FAISS for two (electrician at 0.67 and kitchen appliance technician at 0.73), and was slightly worse for one (sewing machine technician, 0.86 versus 0.88). The largest gains were for:

- well cleaner: +0.23, from 0.64 to 0.87;
- AC technician and roof repairer: +0.17 each;
- gas cooker technician: +0.16;
- ceiling installer: +0.15;
- kitchen water heater: +0.14.

Under the SVM, the strongest categories were tiler (F1 0.90), upholstery specialist, solar panel technician and welder (0.89 each). These trades have distinctive vocabulary such as tiles, sofa covers, panels and inverters, and welding. The weakest were:

- carpenter (0.55);
- electric oven technician, furniture restoration and plumber (0.65 each);
- electrician (0.67);

- bathroom water heater, house cleaner and roof repairer (0.68 each).

Every one of these weak categories has at least one close neighbour in the taxonomy.



Figure 6.2: Experiment A: per-category F1-score of FAISS top-1 search and the SVM classifier, sorted by SVM F1 (dashed lines show the macro averages)

6.1.3 Experiment B: SVM on the cleaned dataset

On the cleaned dataset (about 19,400 rows; test set 3,885 rows with 99 to 100 per category), the SVM reached the results in Table 6.2.

Table 6.2: Experiment B – SVM results on the cleaned dataset (3,885-row test set)

Metric	Macro average	Weighted average
Accuracy	78.35%	78.35%
Precision	0.79	0.79
Recall	0.78	0.78
F1-score	0.78	0.78

The 95% confidence interval for accuracy is [77.05%, 79.65%]. This overlaps almost entirely with the interval for Experiment A. Because the training and test data differ, the two accuracies should not be read as an improvement or a decline. The finding is that the SVM's performance is stable, at about 78–79% accuracy and 0.78–0.79 macro-F1, across two versions of the dataset.

6.1.4 Error analysis

Figure 6.3 shows the row-normalised confusion matrix of the SVM in Experiment A. The diagonal is dominant for every category: no category's true samples are more often assigned to another single category than to itself. The off-diagonal mass is concentrated in a small number of category pairs, shown in Figure 6.4.

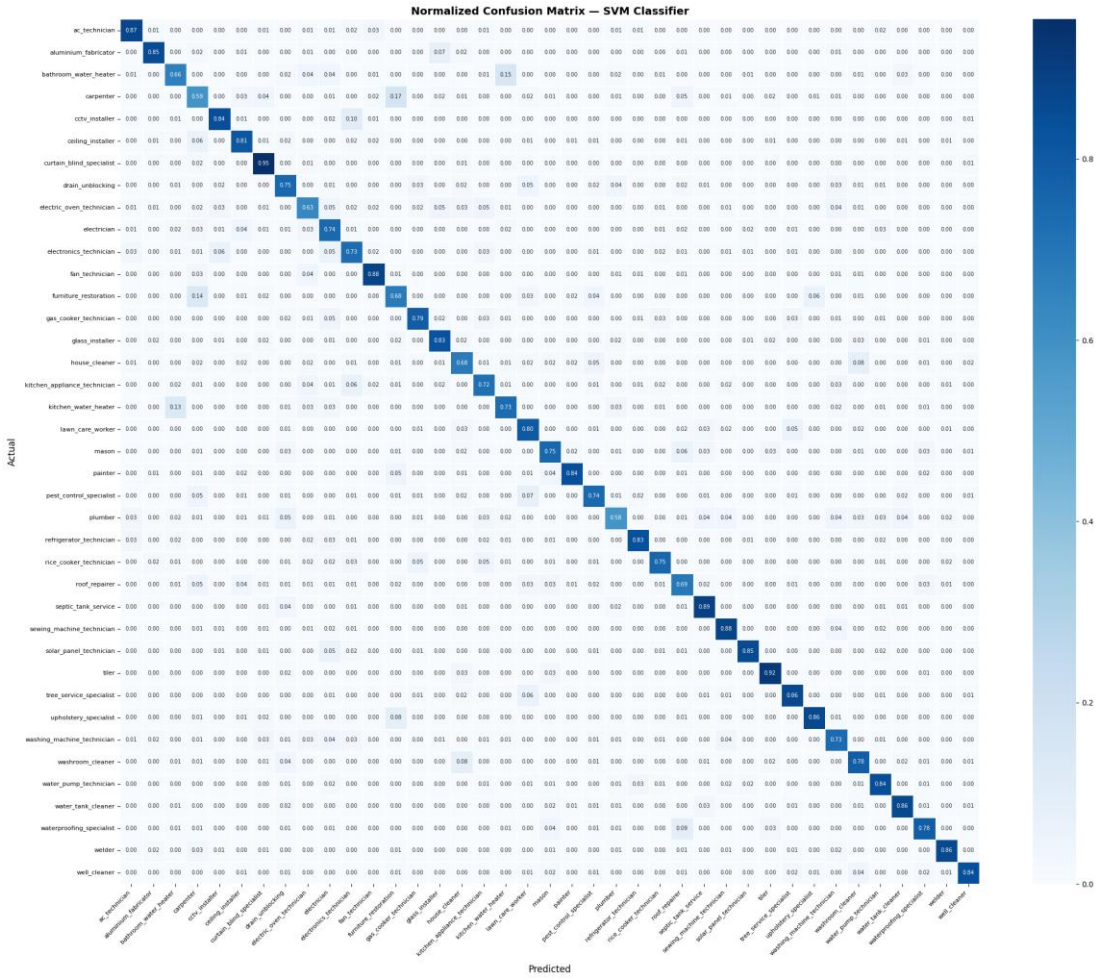


Figure 6.3: Experiment A: row-normalised confusion matrix of the SVM classifier (rows = actual category, columns = predicted category)

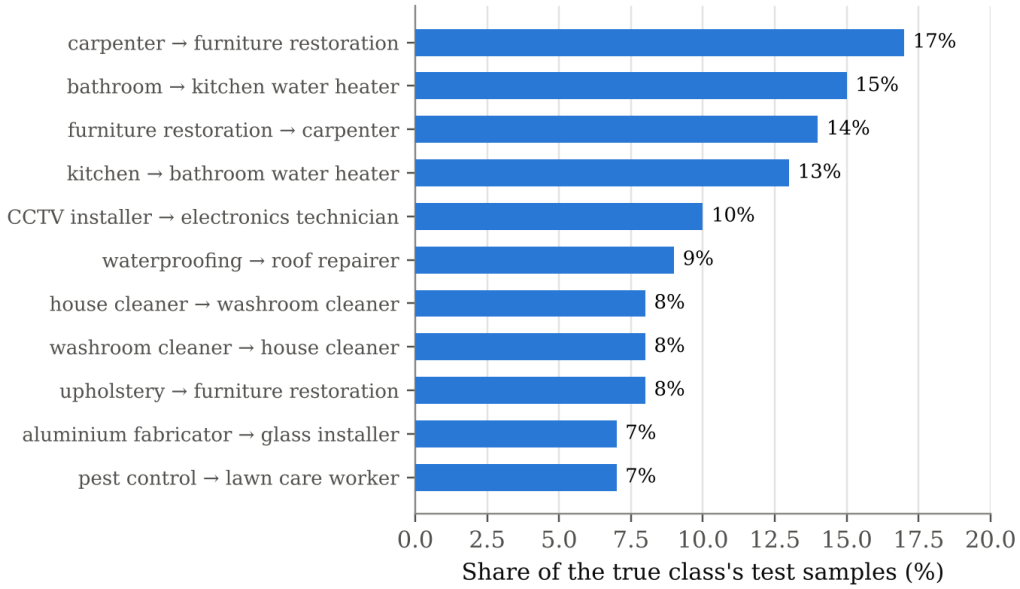


Figure 6.4: Experiment A: most frequent confusions of the SVM classifier, as a share of each true category's test samples

These confusions fall into three groups:

1. **Overlapping trades.** 17% of carpenter descriptions were predicted as furniture restoration and 14% of furniture restoration descriptions as carpenter. Upholstery specialist (8% to furniture restoration) and aluminium fabricator (7% to glass installer) are similar cases. Real customers would often accept either trade for such jobs, for example repairing a wooden cupboard door.
2. **Same appliance, different location.** 15% of bathroom water heater descriptions were predicted as kitchen water heater, and 13% in the opposite direction. The two categories differ mainly in the room mentioned, which is a weak signal when the description talks about the heater itself.
3. **Neighbouring service scopes.** Further confusions occurred between the following pairs:
   - house cleaner and washroom cleaner (8% each way);
   - CCTV installer and electronics technician (10%);
   - waterproofing specialist and roof repairer (9%);
   - pest control specialist and lawn care worker (7%).

The FAISS notebook's error breakdown shows the same pattern in stronger form. Of the 109 carpenter test descriptions, 13 were matched to furniture restoration, and 16 furniture-restoration descriptions were wrongly predicted as carpenter. These results led to the cleaning of ambiguous carpenter rows in Experiment B.

6.2 Product and Technical Results

6.2.1 Functional behaviour

The trained model and label encoder were saved, reloaded through the inference function, and served through the FastAPI service deployed on Microsoft Azure. The service returns a single worker_type label from the 39 categories and a confidence score, as required by FR1 to FR5 (Table 4.1).

6.2.2 Sample queries

Table 6.3 shows the notebook's live inference demonstration on short, realistic queries, most of them romanised Singlish. Eleven of the twelve queries received a top-category probability above 0.85, and their predicted categories are the trades a customer would expect. The exception, "lin sudda karanawa" (cleaning the well), produced a flat distribution, with 0.17 for washroom cleaner, 0.13 for electrician and 0.10 for AC technician. The 0.4 threshold would reject this query instead of sending the customer to a washroom cleaner.

These queries were written by the team and some may be identical or very close to rows in the training data. For example, "watura bata pupurala" also appears in the training set. They therefore illustrate the model's behaviour but are not independent evidence of accuracy.

Table 6.3: Sample queries passed through the trained SVM (notebook output)

Query	Top prediction	Confidence	2nd prediction	Accepted at 0.4?
watura bata pupurala	plumber	89.41%	lawn_care_worker (1.13%)	Yes
current eka wada na	electrician	96.28%	ac_technician (0.83%)	Yes
thnakola kapanna oona	lawn_care_worker	91.52%	welder (1.14%)	Yes
AC eka sisil wenne na	ac_technician	99.98%	electric_oven_technician (0.00%)	Yes

Query	Top prediction	Confidence	2nd prediction	Accepted at 0.4?
panuwa gedara	pest_control_specialist	95.00%	roof_repairer (1.65%)	Yes
lin sudda karanawa	washroom_cleaner	17.38%	electrician (12.63%)	No – rejected
sofa eka aluth kaputa danna	upholstery_specialist	96.67%	furniture_restoration (2.44%)	Yes
kunu kaanu block una	drain_unblocking	93.88%	septic_tank_service (1.70%)	Yes
dora eke frame aluth hadanna timber	carpenter	95.06%	roof_repairer (1.99%)	Yes
geyser eke pressure relief valve drip	bathroom_water_heater	85.83%	kitchen_water_heater (3.96%)	Yes
kussiye sink eke upper instant heater	kitchen_water_heater	97.10%	bathroom_water_heater (2.71%)	Yes
bathroom tile karanna	tiler	94.38%	washroom_cleaner (3.61%)	Yes

The second-ranked categories are informative. For the two water-heater queries, the runner-up is the other water-heater category; for the sofa query, it is furniture restoration. This matches the confusion pairs in Section 6.1.4 and suggests that showing the top two or three categories when confidence is moderate could be a useful enhancement.

6.2.3 Efficiency

On Colab, the SVM trained in 203.6 seconds, and batch encoding took about 0.4–0.5 ms per sentence. The model artefacts are small: a 384-dimensional SVM with 14,120 support vectors, plus the roughly 0.1-billion-parameter embedding model. They run on a CPU without a GPU, which satisfies the intent of NFR2.

6.3 Integration Results

The Flutter application, the Node.js backend, the Component 01 FastAPI service deployed on Azure and Component 02 were connected end to end. A query typed in the application's search bar was sent to the backend, categorised by Component 01, and the resulting worker category was passed to Component 02, which returned the matching workers. When the 0.4 rule rejected a query, the application displayed the message "We couldn't find the worker, try again." Figure 6.5 shows the search screen with a Singlish query, and Figure 6.6 shows the resulting worker list.

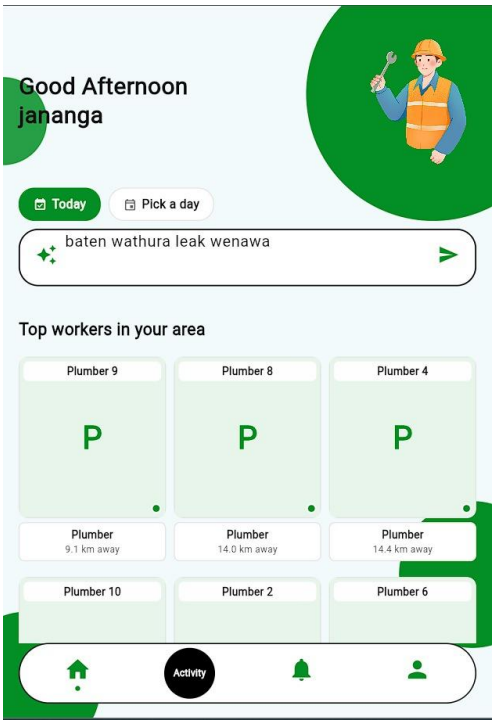


Figure 6.5: Flutter search screen with a Singlish query

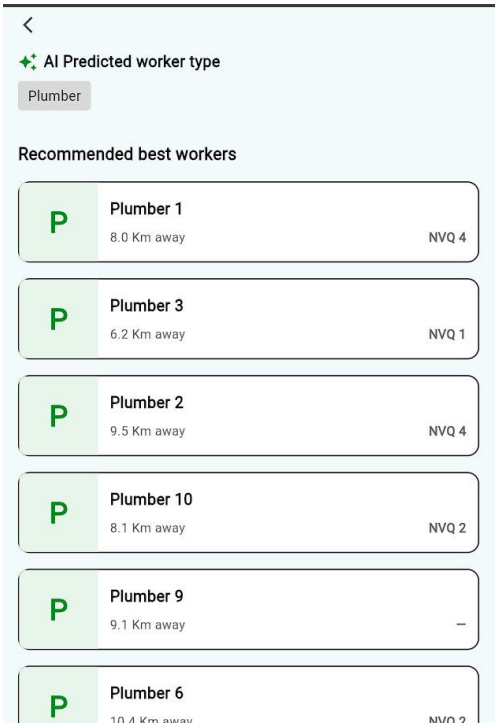


Figure 6.6: The resulting worker list

6.4 Summary of Findings

The findings are summarised below by research question.

- **RQ1.** A pretrained multilingual sentence-embedding model used without fine-tuning provides features that separate 39 household worker categories well above chance. Even the simplest approach on top of it, nearest-neighbour search, reaches 71.50% accuracy.
- **RQ2.** On identical data and features, an RBF-kernel SVM outperforms FAISS top-1 search by 7.18 percentage points in accuracy (78.68% vs 71.50%) and by 0.08 in macro-F1 (0.79 vs 0.71). The confidence intervals do not overlap, and F1 improves in 36 of 39 categories.
- **RQ3.** Errors are concentrated in pairs of closely related categories: carpenter and furniture restoration, the two water-heater categories, the two cleaning categories, CCTV and electronics, and waterproofing and roof repair. These reflect overlap in the taxonomy and in the generated data, not random noise. After cleaning ambiguous carpenter rows (Experiment B), performance remained stable at 78.35% accuracy and 0.78 macro-F1.
- **RQ4.** The model has been packaged as a FastAPI service that returns one worker category with a confidence score and rejects queries below 0.4. It is deployed on Microsoft Azure App Service and called by the Node.js backend, which passes the predicted category to Component 02; the complete search flow was tested end to end from the Flutter application.

7 DISCUSSION

7.1 Interpretation of Findings

Why a multilingual encoder works on Singlish at all. The embedding model was not trained on Sinhala (Section 2.2.3), yet the system handles many romanised Singlish queries confidently. Two mechanisms explain this.

- Singlish household descriptions are rich in English content words: "AC", "tile", "geyser", "pipe", "sink", "sofa", "pump", "block", "valve" and "current". These carry most of the information about the trade, and the model represents them well.
- The SentencePiece tokenizer never fails on an unfamiliar word; it splits it into sub-word pieces. Although these pieces have no learned Sinhala meaning, recurring Singlish words such as "watura", "lin", "gaha" and "biththi" are split in a consistent way. The SVM can then learn from the training data which regions of the embedding space they push a sentence into.

The model therefore does not "understand" Singlish in the way it understands English. The classifier learns the vector patterns that Singlish phrases produce for each trade.

The same reasoning explains the main weakness. Different spellings of the same Sinhala word (watura, wathura, watr) are split into different sub-word pieces and produce vectors that may be only weakly similar. Nothing in the pipeline explicitly maps them together. A query written with a spelling that does not occur in the training data, and with few English anchor words, is likely to receive low confidence. The 0.4 rule then rejects it, which is the intended safe behaviour, or it is assigned to a neighbouring trade.

Why the SVM beats nearest-neighbour search. FAISS top-1 search bases each decision on a single training example. If the closest training sentence happens to be a borderline or mislabelled row, for example a carpenter-like description labelled furniture restoration, the prediction follows it. The SVM uses every training example to place a decision boundary. Isolated noisy points therefore have less influence, and the RBF kernel can carve out category regions of irregular shape.

This explains why the largest SVM gains occurred in categories whose descriptions are spread out in the embedding space, such as well cleaner, AC technician, roof repairer and gas cooker technician. In those categories, a single nearest neighbour is often misleading but the overall region is still distinctive. It also explains why the gains were smallest where two categories overlap genuinely: no boundary can separate examples that mean the same thing.

What the confusion pairs say about the taxonomy. Promoting sub-types to full categories, for example separate bathroom and kitchen water heater technicians, made the output simpler for Component 02. It also created pairs that are hard to distinguish from text alone, and the confusion matrix shows exactly these pairs. Some of these errors may matter little in practice: a furniture restorer can often handle a simple carpentry repair. Others, such as a CCTV problem sent to a general electronics technician, could lead to the wrong worker. The results therefore point to a design choice for the group: merge some categories, or keep them separate and show the top two candidates when they are close.

The 0.4 threshold. The sample queries suggest that the model's probabilities behave sensibly. Clear requests receive very high confidence. A vague request, "lin sudda karanawa", receives a flat distribution with a top probability of only 0.17 and is rejected. Without measured coverage on real queries, however, it is not yet known what share of genuine customer requests would be rejected at 0.4. It is also not known how accurate the accepted predictions are. This is the most important open question for the product.

7.2 Comparison with Literature Baselines and Existing Solutions

Direct numerical comparison with published work is not possible, because no other study uses this task, language mix or dataset. Indirect comparisons are still informative.

- **Intent benchmarks.** Casanueva et al. [26] showed that fixed sentence encoders with a light classifier are strong, efficient intent detectors. The present results support this for a new language mix, with 78.68% accuracy over 39 classes from a model trained on a CPU in under four minutes. Their encoders were evaluated

on English benchmarks with human-written data. The present encoder had never seen Sinhala, and the categories include deliberately fine-grained siblings, so lower absolute accuracy is expected.

- **Help-desk ticket classification.** Paramesh et al. [13] reported 88.23% accuracy for an SVM on 18 IT ticket classes in English. The present task has more than twice as many classes, shorter texts and code-mixed language. It reaches about ten points less accuracy using semantic embeddings instead of TF-IDF. Their finding that an SVM is among the strongest single classifiers is consistent with this study.
- **Sinhala-English code-mixed classification.** Rathnayake et al. [6] and Rizvi et al. [7] fine-tuned multilingual transformers for sentiment and related tasks with few labels. The present work shows that, even without fine-tuning, multilingual embeddings support a much larger label set for a different task (service requests). Fine-tuning, as in those studies, is therefore a promising next step (Section 9.4).
- **Synthetic data.** Sahu et al. [43] found that AI-generated intent data helps least when intents are semantically close, because generated examples drift into neighbouring intents. The confusion pairs observed here (carpenter and furniture restoration, the two water-heater categories) are exactly such close intents. This suggests that part of the residual error comes from label noise in the generated data, not only from model limitations.
- **Existing platforms.** Compared with TaskRabbit, ikman.lk, work.lk and HodaBass.lk (Table 2.1), the component removes the requirement that the customer knows or selects the trade. On the test set it chooses the correct trade for nearly four in five descriptions, and it declines when it is unsure. Existing platforms cannot accept a romanised Singlish problem description at all.

7.3 Research Contribution

The research contributions of this component are:

1. **Problem formulation.** It formulates household service discovery in Sri Lanka as a 39-class intent classification task over romanised Sinhala-English and English text. This task has not been addressed in the literature reviewed.

2. **Dataset.** It constructs a balanced dataset of 21,294 household problem descriptions with about 42% romanised Singlish, together with a cleaned variant. The construction process and limitations are documented.
3. **Controlled comparison.** It provides a controlled comparison, on identical data, split and embeddings, showing that an RBF-kernel SVM improves on FAISS nearest-neighbour search by 7.18 accuracy points and 0.08 macro-F1 for this task.
4. **Error analysis.** It identifies the category pairs that are systematically confused, linking them to taxonomy design and to known weaknesses of AI-generated data.
5. **Evidence on multilingual encoders.** It shows that a multilingual encoder not trained on Sinhala can still support useful classification of romanised Singlish service requests. It also explains the limits of this approach, especially spelling variation.

7.4 Product Contribution

For the marketplace, Component 01 provides the following:

- A free-text search entry point that accepts the way Sri Lankans actually describe household problems, including romanised Singlish.
- A single, well-defined category label that drives Component 02's recommendations and Component 03's category-specific trust scores.
- A confidence score and a rejection rule that keep the application from confidently sending customers to unrelated trades.
- A lightweight, CPU-only microservice that can be retrained and redeployed independently of the other components.

7.5 Limitations and Threats to Validity

Construct validity – synthetic data. Most training and test descriptions were generated by an AI assistant from team-written seeds and Facebook-derived problems, and labels were set at generation time. The test set comes from the same process. The reported accuracy therefore measures how well the model recognises categories in AI-

assisted descriptions. It does not measure performance on real customer queries, which may be shorter, misspelled in different ways, use different vocabulary, or describe several problems at once. This is the most important limitation of the study. It can only be resolved by evaluating on real queries (Section 9.4).

Internal validity – leakage and near-duplicates. The split was stratified and random, but generated data tends to contain near-paraphrases. Five exact duplicate descriptions remain in the file, and the FAISS demonstration found an exact training match (similarity 1.0) for the query "watura bata pupurala". Near-duplicates shared between the training and test sets would make the test easier than real use, so both approaches' accuracies may be optimistic. The comparison between FAISS and SVM is still fair, because both share the same split.

Internal validity – label noise. There was no independent annotation and no inter-annotator agreement measure. Some rows, especially in overlapping trades, may be labelled with a defensible but different category from the one a human expert would choose. Experiment B addressed this only for carpenter.

External validity – language coverage. Sinhala script, Tamil and Tanglish were not included. The model will not reliably handle them, and the system currently serves only English and romanised Singlish users.

Statistical conclusion validity. Each experiment used a single 80/20 split, and no cross-validation was performed. The confidence intervals in Chapter 6 address sampling variation in the test set but not variation due to the choice of split. The SVM hyper-parameters (C = 10, gamma = "scale") were not tuned with a separate validation set. Because they were fixed in advance and not chosen on the test set, this does not bias the reported results, but tuning might improve them.

Unmeasured product qualities. The following remain open:

- the rejection rate and accuracy at the 0.4 threshold on real queries;
- user satisfaction.

Alternatives not tested. Character n-gram TF-IDF, FastText, fine-tuned mBERT, fine-tuned XLM-R and SinBERT were not evaluated. The conclusion that the SVM is the best option therefore holds only relative to the FAISS baseline.

7.6 Implications

For **practitioners** building services for Sri Lankan users, the results show that a usable free-text classifier can be built quickly, without labelled Sinhala corpora, GPUs or transliteration tools. It needs a multilingual sentence encoder and a classical classifier. They also show that the category taxonomy must be designed with the classifier in mind: categories that customers cannot distinguish in words will also be confused by the model.

For **researchers**, the study adds a new task and language mix to code-mixed NLP. It also highlights two open problems: robust representation of unstandardised romanised Sinhala, and the reliability of AI-generated training data for fine-grained intents.

For the **group project**, the error analysis suggests concrete next steps:

- Collect real queries through a future user study.
- Consider merging or relabelling the most confused category pairs.
- Use the second-ranked category when confidence is moderate.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value Proposition and Target Users

Component 01 is not sold on its own. It is the entry point that makes the marketplace easy to use. Its value proposition for each stakeholder is set out below.

- **Customers (households).** "Describe your problem the way you would tell a neighbour, in English or Singlish, and get the right kind of worker." This removes the step that most often fails for customers: knowing which trade to look for. The primary target users are urban and suburban households who use smartphones and messaging apps and who regularly need repairs, installation, cleaning or maintenance. The secondary target users are people who have recently moved, older users who are less familiar with trade terminology, and landlords or property managers who handle many different problems.
- **Workers.** Requests reach workers already matched to their trade, which reduces irrelevant calls and wasted travel. The matching is precise, at the level of 39 categories, so specialists such as water-tank cleaners, solar-panel technicians and sewing-machine technicians receive the jobs they are qualified for.
- **The platform.** A correct category is what allows Components 02, 03 and 04 to work. Better categorisation leads to better recommendations, trust scores and price estimates, and therefore to more completed jobs.

8.2 Market and Competitive Position

At the start of 2025, Sri Lanka had about 23.2 million people, 12.4 million internet users (53.6% penetration), 8.2 million social-media user identities and 29.3 million cellular mobile connections [54]. Much household-service discovery already happens informally on social media and messaging platforms, in exactly the code-mixed, romanised language that this component is designed to understand.

The competitive landscape includes:

- **classified-advertisement sites** such as ikman.lk, which list workers by category and location [18];

- **local freelance marketplaces** such as work.lk [19];
- **worker-advertisement apps such as HodaBass.lk [20];**
- **international task platforms** such as TaskRabbit, which do not operate in Sri Lanka but set user expectations [17];
- **informal word-of-mouth and Facebook groups.**

As Table 2.1 showed, none of these offers free-text understanding of Singlish problem descriptions. The component's competitive advantage is therefore twofold:

- a lower barrier to the first search, which is where customers decide whether an app is useful;
- a proprietary, domain-specific dataset and taxonomy that competitors would need time to reproduce.

8.3 Business and Sustainability Model

The component's costs are low and predictable. It needs no GPU, and the embedding model and SVM run on a modest CPU instance. Inference cost per query is a small fraction of a second of CPU time. The main ongoing cost is human effort to collect real queries, review mislabelled ones and retrain the model periodically.

The marketplace as a whole could generate revenue in the following ways. These are proposals for the group's business model, not tested pricing:

- a small commission on each completed job booked through the platform;
- optional subscription or featured-listing plans for workers and service companies;
- partnerships with hardware stores, appliance brands and service companies, which could receive leads in their categories.

Component 01 supports each of these streams. Correct categorisation turns more searches into bookings and produces anonymised demand data by category and location, for example "drain unblocking requests rise in the monsoon". Such data could be valuable to partners if handled lawfully and in aggregate.

The model is sustained by a data flywheel. Every query, with the category the customer finally booked, can become a new labelled example. Retraining on these real examples steadily reduces the dependence on synthetic data.

8.4 Scalability, Intellectual Property and Compliance

Scalability. The service is stateless. Several copies can run behind a load balancer, and each request is independent. SVM inference cost grows with the number of support vectors (14,120 at present). If the dataset grows to hundreds of thousands of examples, training time and model size will grow considerably. Switching to a linear classifier or a small fine-tuned network on the same embeddings would then be a natural scaling step.

Intellectual property. The pretrained embedding model is distributed under the Apache 2.0 licence [4], which permits commercial use with attribution. scikit-learn, FAISS, sentence-transformers and FastAPI are also open source under permissive licences. The dataset, the 39-category taxonomy and the trained classifier were created by the team and form the component's main proprietary assets. Because part of the dataset was produced with an AI assistant, the group should check the provider's terms of use before commercial release. Facebook content was used only as inspiration and was not copied, so no third-party text is redistributed.

Compliance. The component processes only the text of the problem description. It does not require identity, contact details or location; these are handled by other parts of the system. This keeps it consistent with the data-minimisation and purpose-limitation principles of Sri Lanka's Personal Data Protection Act, No. 9 of 2022 [48]. Before commercial launch, the platform's privacy notice should tell customers that their problem text is processed automatically to find a worker category, and that anonymised queries may be used to improve the model.

8.5 Contribution to the Sustainable Development Goals

The component contributes to the United Nations Sustainable Development Goals [55] in four ways:

- **SDG 8 – Decent work and economic growth.** By matching household jobs to the correct trade, the component helps informal tradespeople receive relevant work through a digital channel and reduces time lost on unsuitable calls.
- **SDG 9 – Industry, innovation and infrastructure.** It applies modern language technology to a traditional, largely offline service sector, and demonstrates a low-cost AI component that a local start-up can operate.
- **SDG 10 – Reduced inequalities.** It accepts the language people actually use, romanised Singlish, instead of requiring formal English trade names. This lowers a language barrier to digital services. Extending it to Sinhala and Tamil scripts (Section 9.4) would widen this inclusion further.
- **SDG 11 – Sustainable cities and communities.** It makes it easier to get timely household repairs, such as water leaks, blocked drains and electrical faults. This supports safer and better-maintained homes.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis set out to let Sri Lankan customers find a household worker by describing their problem in their own words, in English or romanised Sinhala-English, instead of choosing a trade from a list. It built a 39-category dataset of 21,294 problem descriptions, about 42% of them romanised Singlish. It encoded them with a pretrained multilingual sentence-embedding model, and compared two ways of turning the embeddings into a category.

On identical data, split and features, an RBF-kernel SVM reached 78.68% accuracy and 0.79 macro-F1. This was 7.18 accuracy points and 0.08 macro-F1 better than FAISS nearest-neighbour search (71.50%, 0.71), and F1 improved in 36 of 39 categories. After ambiguous carpenter rows were cleaned, a separate experiment confirmed stable performance (78.35% accuracy, 0.78 macro-F1). The remaining errors are concentrated in pairs of closely related trades.

The model was packaged as a FastAPI service with a 0.4 confidence rule deployed on Microsoft Azure, and integrated with the group's Node.js backend, Flutter application and Component 02. The work shows that useful service-intent recognition for romanised Singlish is achievable without Sinhala-specific resources or GPU training. It also shows that the evidence so far comes from largely AI-generated data, so validation on real customer queries is essential before release.

9.2 Achievement of Objectives

Table 9.1 shows how far each objective set in Section 1.5 has been achieved.

Table 9.1: Achievement of research objectives

Objective	Status	Evidence
1. Review literature and identify the gap	Achieved	Chapter 2; gap synthesis in Section 2.5
2. Build a labelled 39-category English/Singlish dataset	Achieved, with limitations	21,294 balanced rows (Section 3.2); mostly AI-generated, no independent annotation

Objective	Status	Evidence
3. Design a pipeline without hand-crafted Singlish rules	Achieved	Pretrained multilingual embeddings + SVM, no text pre-processing (Sections 3.3, 4.4)
4. Compare similarity search and a trained classifier	Achieved	Experiment A: SVM 78.68% vs FAISS 71.50% (Section 6.1.1)
5. Evaluate with accuracy, precision, recall, F1 and error analysis	Achieved	Sections 6.1.1-6.1.4; Appendix A
6. Serve the model with a rejection rule and integrate it	Achieved	FastAPI service deployed on Azure and integrated end to end (Sections 5.3, 5.5, 6.3)

9.3 Individual Research and Product Contributions

On the research side, the author contributed:

- the formulation of household service discovery as a 39-class intent classification task for romanised Sinhala-English and English text;
- a new, documented dataset for this task;
- a controlled comparison of nearest-neighbour search and SVM classification on multilingual sentence embeddings;
- an error analysis that links model confusions to taxonomy design and to the behaviour of AI-generated data.

On the product side, the author delivered:

- a working intent recognition service with a confidence-based rejection rule;
- a single, well-defined output contract (worker_type) that drives the marketplace's recommendation and trust components.

9.4 Future Directions

The following directions would strengthen both the research and the product.

1. **Real-query evaluation set.** Collect several hundred real problem descriptions through a user study and anonymised application logs. Label each one by two people and measure their agreement. Report accuracy separately for English and Singlish. This is the most important next step.

2. **Threshold calibration.** Using the real-query set, plot accuracy against coverage for thresholds from 0.1 to 0.9. Choose the value that gives the best balance between wrong answers and rejected queries, instead of the current default of 0.4.
3. **Spell correction and normalisation layer (planned Phase 2).** Add a Singlish normalisation step that maps common variant spellings to one canonical form before encoding, for example wathura, watr → watura and kussiya, kusiya → kussiya. Possible methods are a curated dictionary, character n-gram similarity, or a learned transliteration model such as the one by Perera et al. [9].
4. **Fine-tuning the encoder.** Fine-tune the multilingual encoder on the household dataset, for example with contrastive sentence-transformer fine-tuning as in SetFit [46], or full fine-tuning of XLM-R [29]. Compare it with untested alternatives: character n-gram TF-IDF, FastText [45] and SinBERT [41].
5. **Taxonomy refinement.** Review the most confused pairs (carpenter and furniture restoration, the two water-heater categories, house cleaner and washroom cleaner) with the group. Decide whether to merge them or to keep them separate and show the top two categories when their probabilities are close.
6. **Top-k suggestions and clarifying questions.** When confidence is between the rejection threshold and a higher "certain" level, show the two or three most likely trades, or ask a short follow-up question, instead of a single answer.
7. **More languages.** Extend the dataset and model to Sinhala script, Tamil script and Tanglish, so that the service is usable by all major language groups in Sri Lanka.
8. **Robust evaluation.** Use stratified k-fold cross-validation and a separate validation set for hyper-parameter tuning. Remove near-duplicate paraphrases across splits, and apply McNemar's test to paired predictions when comparing models.
9. **Monitoring.** Add logging and monitoring to the Azure deployment to track category distributions, rejection rates and drift over time, and restrict access to the endpoint so that only the backend can call it.

REFERENCES

[1] Google, "Flutter - Build apps for any screen." [Online]. Available: https://flutter.dev (accessed Oct. 8, 2026).

[2] S. Ramírez, "FastAPI." [Online]. Available: https://fastapi.tiangolo.com (accessed Oct. 8, 2026).

[3] N. Reimers and I. Gurevych, "Making monolingual sentence embeddings multilingual using knowledge distillation," in Proc. 2020 Conf. Empirical Methods Natural Lang. Process. (EMNLP), Online, 2020, pp. 4512–4525.

[4] Sentence-Transformers, "paraphrase-multilingual-MiniLM-L12-v2," Hugging Face model card. [Online]. Available: https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2 (accessed Oct. 8, 2026).

[5] C. Cortes and V. Vapnik, "Support-vector networks," Mach. Learn., vol. 20, no. 3, pp. 273–297, 1995.

[6] H. Rathnayake, J. Sumanapala, R. Rukshani, and S. Ranathunga, "Adapter-based fine-tuning of pre-trained multilingual language models for code-mixed and code-switched text classification," Knowl. Inf. Syst., vol. 64, no. 7, pp. 1937–1966, 2022.

[7] A. Rizvi, N. Thamindu, A. M. N. H. Adhikari, W. P. U. Senevirathna, D. Kasthurirathna, and L. Abeywardhana, "Enhancing multilingual sentiment analysis with explainability for Sinhala, English, and code-mixed content," arXiv preprint arXiv:2504.13545, 2025.

[8] I. Smith and U. Thayasivam, "Language detection in Sinhala-English code-mixed data," in Proc. 2019 Int. Conf. Asian Lang. Process. (IALP), Shanghai, China, 2019.

[9] S. Perera, L. Prabhath, T. G. D. K. Sumanathilaka, and I. Anuradha, "IndoNLP 2025 shared task: Romanized Sinhala to Sinhala reverse transliteration using BERT," in Proc. 1st Workshop Natural Lang. Process. Indo-Aryan Dravidian Lang., Abu Dhabi, UAE, 2025, pp. 135–140.

[10] L. K. D. Fernando and G. U. Ganegoda, "Resbot: A bi-lingual restaurant booking conversational artificial intelligence," in Proc. Int. Conf. Inf. Technol. Res. (ICITR), Moratuwa, Sri Lanka, Dec. 2023.

[11] P. Avishka, N. Ravikumar, K. Banujan, and H. Gunasinghe, "Towards a more intuitive Sinhala chatbot: Leveraging NLU for enhanced intent identification and entity extraction," Sabaragamuwa Univ. J., vol. 19, no. 2, pp. 46–63, 2023, doi: 10.4038/suslj.v19i2.7792.

[12] Y. Karunanayake, U. Thayasivam, and S. Ranathunga, "Transfer learning based free-form speech command classification for low-resource languages," in

Proc. 57th Annu. Meeting Assoc. Comput. Linguistics: Student Research Workshop, Florence, Italy, 2019, pp. 288-294, doi: 10.18653/v1/P19-2040.

[13] S. P. Paramesh, C. Ramya, and K. S. Shreedhara, "Classifying the unstructured IT service desk tickets using ensemble of classifiers," in Proc. 3rd IEEE Int. Conf. Comput. Syst. Inf. Technol. Sustain. Solutions (CSITSS), Bengaluru, India, 2018, pp. 221–227.

[14] M. Marcuzzo, A. Zangari, M. Schiavinato, L. Giudice, A. Gasparetto, and A. Albarelli, "A multi-level approach for hierarchical ticket classification," in Proc. 8th Workshop Noisy User-generated Text (W-NUT), Gyeongju, Republic of Korea, 2022, pp. 201–214.

[15] S. Banerjee, N. Moghe, S. Arora, and M. M. Khapra, "A dataset for building code-mixed goal oriented conversation systems," in Proc. 27th Int. Conf. Comput. Linguistics (COLING), Santa Fe, NM, USA, 2018, pp. 3766–3780.

[16] S. Khanuja, S. Dandapat, A. Srinivasan, S. Sitaram, and M. Choudhury, "GLUECoS: An evaluation benchmark for code-switched NLP," in Proc. 58th Annu. Meeting Assoc. Comput. Linguistics (ACL), Online, 2020, pp. 3575-3585, doi: 10.18653/v1/2020.acl-main.329.

[17] Taskrabbit, "How do I choose a category for my task?," Taskrabbit Support Center. [Online]. Available: https://support.taskrabbit.com/hc/articles/360049547952 (accessed Oct. 8, 2026).

[18] ikman.lk, "Plumber - Building & construction jobs listings," ikman.lk. [Online]. Available: https://ikman.lk/en/ads/i/colombo-5/building-construction-jobs/plumber (accessed Oct. 8, 2026).

[19] "SLT signs MoU with work.lk," Business Today, Aug. 2019. [Online]. Available: https://businesstoday.lk/slt-signs-mou-with-work-lk/ (accessed Oct. 8, 2026).

[20] Xiteb (Pvt) Ltd., "HodaBass.lk," Apple App Store. [Online]. Available: https://apps.apple.com/us/app/hodabass-lk/id1262278190 (accessed Oct. 28, 2026).

[21] A. S. Doğruöz, S. Sitaram, B. E. Bullock, and A. J. Toribio, "A survey of code-switching: Linguistic and social perspectives for language technologies," in Proc. 59th Annu. Meeting Assoc. Comput. Linguistics and 11th Int. Joint Conf. Natural Lang. Process. (Vol. 1: Long Papers), Online, 2021, pp. 1654–1666, doi: 10.18653/v1/2021.acl-long.131.

[22] G. Winata, A. F. Aji, Z. X. Yong, and T. Solorio, "The decades progress on code-switching research in NLP: A systematic survey on trends and challenges," in Findings Assoc. Comput. Linguistics: ACL 2023, Toronto, Canada, 2023, pp. 2936–2978, doi: 10.18653/v1/2023.findings-acl.185.

[23] N. de Silva, "Survey on publicly available Sinhala natural language processing tools and research," arXiv preprint arXiv:1906.02358, 2019.

[24] P. Joshi, S. Santy, A. Budhiraja, K. Bali, and M. Choudhury, "The state and fate of linguistic diversity and inclusion in the NLP world," in Proc. 58th Annu. Meeting Assoc. Comput. Linguistics (ACL), Online, 2020, pp. 6282–6293.

[25] S. Larson et al., "An evaluation dataset for intent classification and out-of-scope prediction," in Proc. 2019 Conf. Empirical Methods Natural Lang. Process. and 9th Int. Joint Conf. Natural Lang. Process. (EMNLP-IJCNLP), Hong Kong, China, 2019, pp. 1311–1316, doi: 10.18653/v1/D19-1131.

[26] I. Casanueva, T. Temčinas, D. Gerz, M. Henderson, and I. Vulić, "Efficient intent detection with dual sentence encoders," in Proc. 2nd Workshop Natural Lang. Process. Conversational AI, Online, 2020, pp. 38-45, doi: 10.18653/v1/2020.nlp4convai-1.5.

[27] J. FitzGerald et al., "MASSIVE: A 1M-example multilingual natural language understanding dataset with 51 typologically-diverse languages," in Proc. 61st Annu. Meeting Assoc. Comput. Linguistics (Vol. 1: Long Papers), Toronto, Canada, 2023, pp. 4277-4302, doi: 10.18653/v1/2023.acl-long.235.

[28] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. 2019 Conf. North Amer. Chapter Assoc. Comput. Linguistics (NAACL-HLT), Minneapolis, MN, USA, 2019, pp. 4171–4186.

[29] A. Conneau et al., "Unsupervised cross-lingual representation learning at scale," in Proc. 58th Annu. Meeting Assoc. Comput. Linguistics (ACL), Online, 2020, pp. 8440–8451.

[30] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. 2019 Conf. Empirical Methods Natural Lang. Process. and 9th Int. Joint Conf. Natural Lang. Process. (EMNLP-IJCNLP), Hong Kong, China, 2019, pp. 3982-3992.

[31] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 5776-5788.

[32] T. Kudo and J. Richardson, "SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing," in Proc. 2018 Conf. Empirical Methods Natural Lang. Process.: System Demonstrations, Brussels, Belgium, 2018, pp. 66-71.

[33] T. Joachims, "Text categorization with support vector machines: Learning with many relevant features," in Proc. 10th European Conf. Machine Learning

(ECML), Lecture Notes in Computer Science, vol. 1398, Chemnitz, Germany, 1998, pp. 137–142.

[34] C.-C. Chang and C.-J. Lin, "LIBSVM: A library for support vector machines," ACM Trans. Intell. Syst. Technol., vol. 2, no. 3, Art. no. 27, 2011.

[35] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.

[36] J. C. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in Advances in Large Margin Classifiers, A. J. Smola, P. Bartlett, B. Schölkopf, and D. Schuurmans, Eds. Cambridge, MA, USA: MIT Press, 1999, pp. 61-74.

[37] T.-F. Wu, C.-J. Lin, and R. C. Weng, "Probability estimates for multi-class classification by pairwise coupling," J. Mach. Learn. Res., vol. 5, pp. 975-1005, 2004.

[38] D. Hendrycks and K. Gimpel, "A baseline for detecting misclassified and out-of-distribution examples in neural networks," in Proc. 5th Int. Conf. Learning Representations (ICLR), Toulon, France, 2017.

[39] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," IEEE Trans. Big Data, vol. 7, no. 3, pp. 535-547, 2021.

[40] I. Smith and U. Thayasivam, "Sinhala-English code-mixed data analysis: A review on data collection process," in Proc. 19th Int. Conf. Advances in ICT for Emerging Regions (ICTer), Colombo, Sri Lanka, 2019.

[41] V. Dhananjaya, P. Demotte, S. Ranathunga, and S. Jayasena, "BERTifying Sinhala - A comprehensive analysis of pre-trained language models for Sinhala text classification," in Proc. 13th Lang. Resour. Eval. Conf. (LREC), Marseille, France, 2022, pp. 7377–7385.

[42] F. Feng, Y. Yang, D. Cer, N. Arivazhagan, and W. Wang, "Language-agnostic BERT sentence embedding," in Proc. 60th Annu. Meeting Assoc. Comput. Linguistics (Vol. 1: Long Papers), Dublin, Ireland, 2022, pp. 878-891.

[43] G. Sahu, P. Rodriguez, I. Laradji, P. Atighehchian, D. Vazquez, and D. Bahdanau, "Data augmentation for intent classification with off-the-shelf large language models," in Proc. 4th Workshop Natural Lang. Process. Conversational AI, Dublin, Ireland, 2022, pp. 47-57, doi: 10.18653/v1/2022.nlp4convai-1.5.

[44] J. Wei and K. Zou, "EDA: Easy data augmentation techniques for boosting performance on text classification tasks," in Proc. 2019 Conf. Empirical Methods Natural Lang. Process. and 9th Int. Joint Conf. Natural Lang. Process. (EMNLP-IJCNLP), Hong Kong, China, 2019, pp. 6382-6388.

[45] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, "Enriching word vectors with subword information," Trans. Assoc. Comput. Linguistics, vol. 5, pp. 135-146, 2017.

[46] L. Tunstall et al., "Efficient few-shot learning without prompts," arXiv preprint arXiv:2209.11055, 2022.

[47] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," Inf. Process. Manage., vol. 45, no. 4, pp. 427-437, 2009.

[48] Parliament of the Democratic Socialist Republic of Sri Lanka, Personal Data Protection Act, No. 9 of 2022. Colombo, Sri Lanka: Government Publications Bureau, 2022.

[49] T. Gebru et al., "Datasheets for datasets," Commun. ACM, vol. 64, no. 12, pp. 86–92, 2021.

[50] M. Mitchell et al., "Model cards for model reporting," in Proc. Conf. Fairness, Accountability, and Transparency (FAT*), Atlanta, GA, USA, 2019, pp. 220–229.

[51] Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE) – Product Quality Model, ISO/IEC 25010:2023, International Organization for Standardization, Geneva, Switzerland, 2023.

[52] Software and Systems Engineering – Software Testing – Part 1: General Concepts, ISO/IEC/IEEE 29119-1:2022, International Organization for Standardization, Geneva, Switzerland, 2022.

[53] National Institute of Standards and Technology, "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," NIST AI 100-1, Gaithersburg, MD, USA, Jan. 2023.

[54] S. Kemp, "Digital 2025: Sri Lanka," DataReportal, Mar. 3, 2025. [Online]. Available: https://datareportal.com/reports/digital-2025-sri-lanka (accessed Oct. 8, 2026).

[55] United Nations General Assembly, "Transforming our world: The 2030 Agenda for Sustainable Development," Resolution A/RES/70/1, New York, NY, USA, Oct. 2015.

GLOSSARY

Term	Definition
Code-mixing	The use of words, phrases or grammar from two languages within one sentence or utterance, e.g. Sinhala and English in "pipe eka leak wenawa".
Confusion matrix	A table showing, for each actual category, how many test samples were predicted as each category; the diagonal holds the correct predictions.
Embedding (sentence embedding)	A fixed-length vector of numbers that represents the meaning of a sentence, such that sentences with similar meaning have similar vectors.
Intent classification	Assigning a user's utterance or query to one of a predefined set of goals or categories.
Macro average	The unweighted mean of a metric over all categories, giving each category equal importance.
Nearest-neighbour search	Classifying a query by finding the most similar stored example and copying its label.
Platt scaling	A method that converts SVM decision values into probabilities by fitting a sigmoid function.
Romanised Sinhala (Singlish)	Sinhala written with Latin (English) letters, typically mixed with English words, with no standard spelling.
Rejection threshold	The minimum confidence required before the component returns a category; below it, the query is declined.
Sub-word tokenisation	Splitting text into frequent word pieces, so that unknown or misspelled words can still be represented by known pieces.
Support vector	A training example that lies on or within the margin of an SVM decision boundary and therefore defines that boundary.

APPENDICES

Appendix A: Category List and Extended Results

This appendix provides the category list and the full per-category results referred to in Chapters 3 and 6.

Table A.1: The 39 worker categories and the estimated share of romanised Singlish rows per category (balanced dataset, 546 rows each)

#	Label (worker_type)	Readable name	Singlish share
1	ac_technician	AC technician	50.7%
2	aluminium_fabricator	Aluminium fabricator	41.8%
3	bathroom_water_heater	Bathroom water heater	50.0%
4	carpenter	Carpenter	52.7%
5	cctv_installer	CCTV installer	44.7%
6	ceiling_installer	Ceiling installer	44.9%
7	curtain_blind_specialist	Curtain blind specialist	48.4%
8	drain_unblocking	Drain unblocking	45.2%
9	electric_oven_technician	Electric oven technician	43.8%
10	electrician	Electrician	42.3%
11	electronics_technician	Electronics technician	48.5%
12	fan_technician	Fan technician	40.1%
13	furniture_restoration	Furniture restoration	51.1%
14	gas_cooker_technician	Gas cooker technician	46.5%
15	glass_installer	Glass installer	29.9%
16	house_cleaner	House cleaner	35.5%
17	kitchen_appliance_technician	Kitchen appliance technician	41.4%
18	kitchen_water_heater	Kitchen water heater	37.0%
19	lawn_care_worker	Lawn care worker	38.1%
20	mason	Mason	46.7%
21	painter	Painter	41.4%
22	pest_control_specialist	Pest control specialist	37.4%
23	plumber	Plumber	48.4%
24	refrigerator_technician	Refrigerator technician	46.5%
25	rice_cooker_technician	Rice cooker technician	39.2%
26	roof_repairer	Roof repairer	39.4%
27	septic_tank_service	Septic tank service	41.4%
28	sewing_machine_technician	Sewing machine technician	38.1%
29	solar_panel_technician	Solar panel technician	33.0%

#	Label (worker_type)	Readable name	Singlish share
30	tiler	Tiler	36.6%
31	tree_service_specialist	Tree service specialist	36.4%
32	upholstery_specialist	Upholstery specialist	29.1%
33	washing_machine_technician	Washing machine technician	36.3%
34	washroom_cleaner	Washroom cleaner	45.8%
35	water_pump_technician	Water pump technician	35.2%
36	water_tank_cleaner	Water tank cleaner	36.4%
37	waterproofing_specialist	Waterproofing specialist	33.0%
38	welder	Welder	35.7%
39	well_cleaner	Well cleaner	57.3%

Table A.2: Experiment A: per-category precision (P), recall (R) and F1-score of FAISS top-1 search and the SVM classifier (test support 109–110 per category)

Category	FAISS P	FAISS R	FAISS F1	SVM P	SVM R	SVM F1
ac_technician	0.68	0.72	0.70	0.87	0.87	0.87
aluminium_fabricator	0.82	0.76	0.79	0.89	0.85	0.87
bathroom_water_heater	0.61	0.50	0.55	0.71	0.66	0.68
carpenter	0.43	0.41	0.42	0.52	0.59	0.55
cctv_installer	0.83	0.79	0.81	0.84	0.84	0.84
ceiling_installer	0.63	0.68	0.65	0.80	0.81	0.80
curtain_blind_specialist	0.84	0.84	0.84	0.81	0.95	0.88
drain_unblocking	0.64	0.65	0.64	0.69	0.75	0.72
electric_oven_technician	0.59	0.61	0.60	0.68	0.63	0.65
electrician	0.70	0.64	0.67	0.60	0.74	0.67
electronics_technician	0.70	0.65	0.67	0.67	0.73	0.70
fan_technician	0.80	0.76	0.78	0.86	0.88	0.87
furniture_restoration	0.63	0.54	0.58	0.63	0.68	0.65
gas_cooker_technician	0.70	0.62	0.66	0.85	0.79	0.82
glass_installer	0.71	0.88	0.79	0.81	0.83	0.82
house_cleaner	0.63	0.66	0.64	0.69	0.68	0.68
kitchen_appliance_technician	0.76	0.70	0.73	0.75	0.72	0.73
kitchen_water_heater	0.54	0.67	0.60	0.74	0.73	0.74
lawn_care_worker	0.71	0.77	0.74	0.72	0.80	0.76
mason	0.66	0.70	0.68	0.77	0.75	0.76
painter	0.78	0.75	0.77	0.91	0.84	0.87
pest_control_specialist	0.76	0.65	0.70	0.77	0.74	0.76
plumber	0.60	0.55	0.58	0.74	0.58	0.65

Category	FAISS P	FAISS R	FAISS F1	SVM P	SVM R	SVM F1
refrigerator_technician	0.84	0.82	0.83	0.91	0.83	0.87
rice_cooker_technician	0.78	0.71	0.74	0.85	0.75	0.80
roof_repairer	0.57	0.46	0.51	0.66	0.69	0.68
septic_tank_service	0.71	0.78	0.75	0.84	0.89	0.86
sewing_machine_technician	0.89	0.87	0.88	0.83	0.88	0.86
solar_panel_technician	0.83	0.84	0.83	0.92	0.85	0.89
tiler	0.80	0.81	0.80	0.88	0.92	0.90
tree_service_specialist	0.77	0.81	0.79	0.85	0.86	0.85
upholstery_specialist	0.80	0.90	0.85	0.91	0.86	0.89
washing_machine_technician	0.68	0.72	0.70	0.75	0.73	0.74
washroom_cleaner	0.71	0.74	0.73	0.75	0.78	0.76
water_pump_technician	0.77	0.83	0.80	0.81	0.84	0.83
water_tank_cleaner	0.70	0.79	0.74	0.88	0.86	0.87
waterproofing_specialist	0.71	0.83	0.77	0.83	0.78	0.81
welder	0.88	0.84	0.86	0.91	0.86	0.89
well_cleaner	0.67	0.61	0.64	0.91	0.84	0.87
Macro average	0.72	0.71	0.71	0.79	0.79	0.79

Appendix B: Academic Integrity and AI Use Disclosure

The following generative AI tools and automated assistance were used in this research. The author remains responsible for the accuracy, originality, citations, code, data, analyses and conclusions in this thesis.

Table B.1: Disclosure of AI tool use

Tool	Purpose	How it was used	Verification performed	Sections / artefacts affected
Claude (Anthropic) – dataset generation	Generating training and test descriptions	Prompted with team-written seed phrases and problems re-written from public Facebook posts to generate English and romanised Singlish descriptions for each of the 39 categories; labels assigned per prompt	Manual removal of duplicates and obviously wrong or ambiguous rows (including the carpenter clean-up for Experiment B); no independent annotation	Dataset (Section 3.2); all reported results (Chapter 6)
Claude (Anthropic) – writing assistance	Structuring and drafting thesis text, literature search support, figure generation from notebook outputs	Drafts produced from the author's notebooks, dataset, answers and the module template; reference details verified against publisher pages	Reviewed by the author; citations and figures checked against sources and notebooks	This thesis document